

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



CHUYÊN ĐỀ AN TOÀN HỆ THỐNG THÔNG TIN

**TÌM HIỂU MÔ HÌNH PHÁT HIỆN
TRANG WEB LỪA ĐẢO DỰA TRÊN
ĐẶC TRƯNG LAI BẰNG KỸ THUẬT HỌC MÁY**

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:

Nguyễn Đức Chiến – AT190107
Lớp: AT19AT

Người hướng dẫn:

TS. Vũ Thị Vân
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2025

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



CHUYÊN ĐỀ AN TOÀN HỆ THỐNG THÔNG TIN

**TÌM HIỂU MÔ HÌNH PHÁT HIỆN
TRANG WEB LỪA ĐẢO DỰA TRÊN
ĐẶC TRƯNG LAI BẰNG KỸ THUẬT HỌC MÁY**

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:

Nguyễn Đức Chiến – AT190107
Lớp: AT19AT

Người hướng dẫn:

TS. Vũ Thị Vân
Khoa An toàn thông tin – Học viện Kỹ thuật mật mã

Hà Nội, 2025

MỤC LỤC

Danh mục kí hiệu và viết tắt.....	iii
Danh mục hình vẽ.....	iv
Danh mục bảng.....	v
Lời cảm ơn.....	vi
Lời nói đầu.....	vii
Chương 1. Tổng quan về phát hiện trang web lừa đảo và kỹ thuật học máy.....	1
1.1. Tổng quan về phát hiện trang web lừa đảo	1
1.1.1. Khái niệm về trang web lừa đảo.....	1
1.1.2. Các phương pháp phát hiện trang web lừa đảo	2
1.2. Giới thiệu về kỹ thuật học máy	4
1.2.1. Khái niệm học máy.....	4
1.2.2. Các phương pháp học máy.....	4
1.2.3. Quy trình cơ bản của học máy.....	6
1.3. Kết luận chương 1	8
Chương 2. Mô hình phát hiện trang web lừa đảo dựa trên đặc trưng lai bằng kỹ thuật học máy.....	9
2.1. Xây dựng bộ dữ liệu.....	9
2.2. Lựa chọn đặc trưng.....	10
2.2.1. Đặc trưng dựa trên URL.....	11
2.2.2. Đặc trưng dựa trên siêu liên kết.....	16
2.2.3. Đặc trưng lại.....	20
2.3. Mô hình phát hiện	20
2.3.1. Lý do lựa chọn thuật toán XGBoost.....	20
2.3.2. Tổng quan thuật toán XGBoost.....	21
2.3.3. Quy trình xây dựng mô hình phát hiện	22
2.3.4. Ưu điểm của mô hình được đề xuất.....	23
2.4. Phương pháp đánh giá.....	24
2.5. Kết luận chương 2	25
Chương 3. Triển khai thực nghiệm.....	26
3.1. Kịch bản	26
3.2. Môi trường thực nghiệm	26
3.3. Mô tả cấu trúc thư mục hệ thống.....	27
3.4. Triển khai huấn luyện mô hình XGBoost	29
3.4.1. Chuẩn bị và chia tập dữ liệu.....	29
3.4.2. Huấn luyện và tối ưu mô hình XGBoost.....	30
3.5. Đánh giá mô hình.....	31
3.6. Ứng dụng web mô phỏng.....	32
3.7. Kết luận chương 3	35
Kết luận.....	36
Tài liệu tham khảo.....	37
Phụ lục.....	38

Phụ lục A: Mã nguồn trích xuất đặc trưng URL.....	38
Phụ lục B: Mã nguồn trích xuất đặc trưng siêu liên kết (hyperlink).....	40
Phụ lục C: Mã nguồn huấn luyện mô hình.....	44
Phụ lục D: Mã nguồn đánh giá mô hình.....	46

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

AI	Artificial Intelligence
API	Application Programming Interface
DNS	Domain Name System
DOM	Document Object Model
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
ML	Machine Learning
SFH	Server Form Handler
SVM	Support Vector Machine
URL	Uniform Resource Locator
XGBoost	Extreme Gradient Boosting

DANH MỤC HÌNH VẼ

Hình 1.1. Cách thức lừa đảo bằng URL Phishing.....	1
Hình 1.2. Mô hình học máy có giám sát.....	5
Hình 1.3. Tổng quan về quy trình học máy.....	7
Hình 2.1. Tỷ lệ tập dữ liệu được sử dụng.....	9
Hình 2.2. Các thành phần của một URL.....	11
Hình 2.3. Cây DOM HTML.....	16
Hình 2.4. Minh họa mô hình XGBoost dựa trên tập hợp các cây quyết định.....	22
Hình 2.5. Mô tả quy trình của phương pháp được đề xuất.....	23
Hình 3.1. Cấu trúc thư mục hệ thống.....	27
Hình 3.2. Giao diện web khi dự đoán URL hợp pháp.....	33
Hình 3.3. Giao diện web khi dự đoán URL lừa đảo.....	34

DANH MỤC BẢNG

Bảng 2.1. Các đặc trưng phân loại được áp dụng.....	10
Bảng 3.1. Ma trận nhầm lẫn.....	31
Bảng 3.2. Đánh giá hiệu suất mô hình.....	32

LỜI CẢM ƠN

Trong quá trình thực hiện Chuyên đề An toàn hệ thống thông tin này, em đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn là TS. Vũ Thị Vân – Giảng viên Khoa An toàn thông tin Học viện Kỹ thuật Mật mã, người đã tận tâm hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện Chuyên đề. Cô không chỉ cung cấp những định hướng quý giá mà còn dành thời gian trao đổi, giải đáp những thắc mắc, giúp em vượt qua những khó khăn và thách thức. Những lời khuyên thiết thực cùng sự tận tình của cô đã giúp em hiểu sâu hơn về vấn đề nghiên cứu, cũng như hoàn thiện tốt đề tài của mình.

Xin cảm ơn tất cả mọi người đã tạo những điều kiện tốt nhất để em có thể hoàn thành Chuyên đề này!

SINH VIÊN THỰC HIỆN CHUYÊN ĐỀ

Nguyễn Đức Chiến

LỜI NÓI ĐẦU

Trong bối cảnh số lượng người dùng Internet ngày càng tăng, các trang web lừa đảo (phishing websites) đang xuất hiện với tần suất ngày càng nhiều và gây ra nhiều rủi ro nghiêm trọng đối với an toàn thông tin người dùng. Các trang web này thường được thiết kế giả mạo giao diện của những dịch vụ hợp pháp nhằm đánh cắp thông tin cá nhân, thông tin đăng nhập hoặc dữ liệu nhạy cảm.

Các giải pháp phòng chống truyền thống như danh sách đen (blacklist) hoặc danh sách trắng (whitelist), phương pháp heuristic (kinh nghiệm) và phương pháp dựa trên sự tương đồng giao diện (visual similarity) vẫn còn nhiều hạn chế: khó phát hiện được các cuộc tấn công lừa đảo mới xuất hiện (zero-day), độ chính xác thấp và phụ thuộc vào nguồn bên thứ ba.

Vì vậy, em thực hiện tìm hiểu mô hình phát hiện trang web lừa đảo dựa trên đặc trưng lai (hybrid features) kết hợp URL-based features và hyperlink-based features bằng kỹ thuật học máy (Machine Learning), nhằm nâng cao độ chính xác, giảm độ trễ trong quá trình phát hiện, loại bỏ sự phụ thuộc vào các dịch vụ bên thứ ba và hướng tới xây dựng một giải pháp phát hiện các trang web lừa đảo hiệu quả.

Chuyên đề được thực hiện nhằm mục đích tìm hiểu giải pháp phát hiện trang web lừa đảo dựa trên đặc trưng lai bằng kỹ thuật học máy.

Mục tiêu đặt ra khi thực hiện Chuyên đề là:

1. Hệ thống lại kiến thức về trang web lừa đảo;
2. Hiểu rõ được quy trình hoạt động của học máy trong việc xây dựng mô hình phát hiện trang web lừa đảo ;
3. Xây dựng và phân tích mô hình phát hiện trang web lừa đảo dựa trên đặc trưng lai (URL + Hyperlink).

Sau thời gian khoảng hai tháng thực hiện Chuyên đề, các mục tiêu về cơ bản đã đạt được. Tuy nhiên học máy là lĩnh vực phức tạp, thời gian thực hiện Chuyên đề tương đối ngắn nên chắc chắn không tránh khỏi thiếu sót. Rất mong được sự góp ý của các thầy cô, cũng như các bạn học viên để Chuyên đề này được hoàn thiện hơn.

SINH VIÊN THỰC HIỆN CHUYÊN ĐỀ

Nguyễn Đức Chiến

CHƯƠNG 1. TỔNG QUAN VỀ PHÁT HIỆN TRANG WEB LỪA ĐẢO VÀ KỸ THUẬT HỌC MÁY

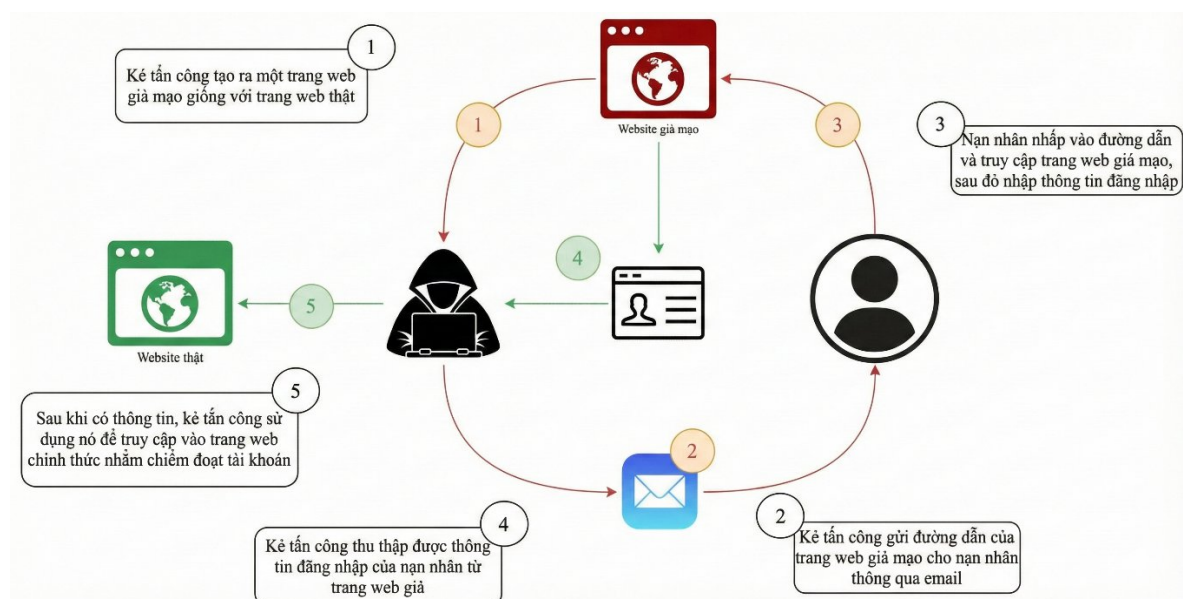
1.1. Tổng quan về phát hiện trang web lừa đảo

1.1.1. Khái niệm về trang web lừa đảo

Trang web lừa đảo (phishing website) là các trang web được tạo ra nhằm giả mạo các trang web hợp pháp để đánh cắp thông tin nhạy cảm của người dùng. Các trang này thường được phát tán thông qua các phương tiện điện tử như email, mạng xã hội hoặc tin nhắn SMS. Một trong những hình thức phổ biến nhất là sử dụng các liên kết giả mạo (phishing URLs), nơi kẻ tấn công thiết kế các địa chỉ URL có cấu trúc tương tự với các trang web hợp pháp nhằm đánh lừa người dùng truy cập và cung cấp dữ liệu cá nhân.

Có năm nguyên nhân phổ biến khiến người dùng rơi vào bẫy của những trang web lừa đảo:

- Người dùng không có hiểu biết sâu sắc về các URL.
- Người dùng không chắc chắn về những trang web mà họ nên tin cậy.
- Do việc chuyển hướng hoặc các URL ẩn, người dùng không thể nhìn thấy toàn bộ địa chỉ của trang web.
- Người dùng không có nhiều thời gian để kiểm tra URL hoặc vô thức truy cập vào một số trang web nhất định.
- Người dùng không thể phân biệt được giữa trang web hợp pháp và trang web lừa đảo.



Hình 1.1. Cách thức lừa đảo bằng URL Phishing

1.1.2. Các phương pháp phát hiện trang web lừa đảo

Hiện nay, nhiều giải pháp chống các trang web lừa đảo đã được nghiên cứu và đề xuất. Các phương pháp phát hiện này được chia thành hai nhóm chính: một là dựa trên việc nâng cao nhận thức và giáo dục người dùng, hai là dựa trên các công cụ phần mềm.

a) Nâng cao nhận thức và giáo dục người dùng

Phương pháp giáo dục người dùng chủ yếu phụ thuộc vào mức độ hiệu quả của việc nâng cao nhận thức cho người dùng Internet, giúp họ hiểu rõ đặc điểm của các cuộc tấn công lừa đảo. Khi người dùng nắm được những kiến thức nền tảng này, họ có thể chủ động nhận diện và tránh các trang web hoặc email lừa đảo.

Để nâng cao hiệu quả phòng chống lừa đảo, nhiều phương pháp giáo dục người dùng đã được đề xuất nhằm trang bị kiến thức nhận diện các dấu hiệu tấn công.

Một trong những hướng tiếp cận nổi bật là áp dụng mô hình trò chơi hóa (gamification). Điển hình là việc phát triển các trò chơi tương tác như 'Anti-Phishing Phil', nơi người dùng được trang bị các khái niệm về phishing thông qua môi trường giả lập. Các kết quả thực nghiệm đã chứng minh nhóm người dùng tham gia trò chơi có tỷ lệ nhận diện trang web giả mạo chính xác hơn đáng kể so với nhóm đối chứng. Hiệu quả của phương pháp này còn được củng cố bởi các khung thiết kế trò chơi dựa trên Lý thuyết Tránh Mọi đe dọa Công nghệ (Technology Threat Avoidance Theory – TTAT), giúp tác động tích cực đến hành vi phòng tránh của người dùng trước các trang web lừa đảo.

Song song với đó, việc đào tạo kỹ năng phân tích URL ngay trong môi trường email cũng được chú trọng. Các nghiên cứu chỉ ra rằng việc giáo dục người dùng nhận biết các dấu hiệu bất thường trong đường dẫn đóng vai trò như một lớp phòng thủ bổ trợ thiết yếu bên cạnh các hệ thống phát hiện tự động.

b) Công cụ phần mềm

Các kỹ thuật phát hiện dựa trên phần mềm được phân loại thành ba nhóm: dựa trên danh sách (blacklist/whitelist), dựa trên độ tương đồng giao diện (visual-similarity), và các phương pháp dựa học máy (machine learning).

- **Kỹ thuật dựa trên danh sách:** Các phương pháp phát hiện dựa trên danh sách được chia thành hai loại: danh sách đen (blacklist) và danh sách trắng (whitelist). Hầu hết các trình duyệt đều duy trì danh sách riêng gồm các URL bị chặn và các URL an toàn. Cơ sở dữ liệu chứa

các URL bị chặn được gọi là blacklist, trong khi cơ sở dữ liệu chứa các URL được phép truy cập hoặc an toàn được gọi là whitelist. Trong phương pháp blacklist, một tên miền đáng ngờ sẽ được kiểm tra xem nó có trùng khớp với các tên miền nằm trong danh sách đen hay không; nếu trùng khớp, trang web sẽ bị phân loại là lừa đảo, ngược lại sẽ được xem là hợp pháp. Phương pháp whitelist thì hoạt động hoàn toàn ngược lại so với blacklist. Tuy nhiên, nhược điểm lớn nhất của các kỹ thuật dựa trên danh sách là không thể phân loại chính xác những trang web lừa đảo mới được tạo ra, đặc biệt là các trang web có thời gian tồn tại chưa đến một ngày, được gọi là các cuộc tấn công phishing zero-day.

- **Kỹ thuật dựa trên độ tương đồng giao diện:** Trong các kỹ thuật dựa trên độ tương đồng giao diện (visual-similarity), giao diện trực quan của trang web đáng ngờ sẽ được so sánh với trang web hợp pháp tương ứng. Để tính toán mức độ tương đồng giữa các trang web, các kỹ thuật này sử dụng nhiều loại đặc trưng khác nhau như mã nguồn trang, hình ảnh, nội dung văn bản, mã HTML, CSS, logo của trang web và nhiều yếu tố khác. Thông thường, kẻ tấn công sao chép gần như toàn bộ giao diện của trang web mục tiêu để người dùng dễ bị đánh lừa. Vì vậy, trong kỹ thuật này, một điểm số tương đồng sẽ được tính cho trang web nghi vấn; nếu điểm số vượt quá một ngưỡng xác định, trang web sẽ bị phân loại là lừa đảo. Tuy nhiên, phương pháp này cũng không thực sự hiệu quả trong việc phát hiện các cuộc tấn công zero-day.
- **Phương pháp dựa trên học máy:** Trong phương pháp dựa trên học máy (Machine Learning), một tập dữ liệu được xây dựng từ các đặc trưng đã được trích xuất. Sau đó, một thuật toán phân loại được huấn luyện trên tập đặc trưng này; trang web sẽ được phân loại là lừa đảo nếu nó khớp với tập đặc trưng đã được mô hình học trước đó. Các phương pháp học máy thậm chí có thể phát hiện các cuộc tấn công zero-day nếu mô hình được huấn luyện với các đặc trưng mang tính suy luận (heuristic features). Trong số các phương pháp phát hiện trang web lừa đảo, học máy được xem là tiếp cận hiệu quả nhất. Tuy nhiên, để thu thập và tính toán đặc trưng từ nhiều nguồn khác nhau như thanh địa chỉ (URL), mã nguồn trang, lưu lượng truy cập web,

DNS, cơ sở dữ liệu WHOIS, công cụ tìm kiếm,... một số giải pháp học máy đòi hỏi khả năng tính toán lớn.

1.2. Giới thiệu về kỹ thuật học máy

1.2.1. Khái niệm học máy

Học máy (Machine Learning - ML) là một lĩnh vực của trí tuệ nhân tạo (AI), sử dụng các thuật toán được đào tạo trên các tập dữ liệu để tạo ra các mô hình tự học có khả năng dự đoán kết quả và phân loại thông tin mà không cần sự can thiệp của con người.

Học máy ngày nay được ứng dụng cho nhiều lĩnh vực khác nhau như gợi ý sản phẩm cho người tiêu dùng, dự đoán biến động thị trường chứng khoán hay dịch văn bản tự động.

1.2.2. Các phương pháp học máy

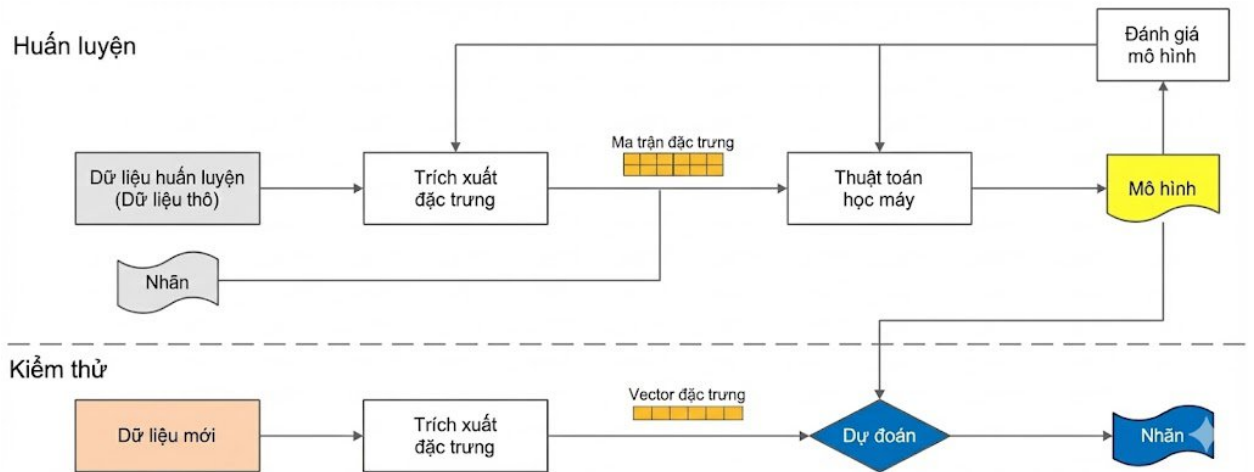
Học máy thường được phân loại theo cách một thuật toán học để trở nên chính xác hơn trong các dự đoán của nó. Có bốn cách tiếp cận cơ bản: học có giám sát, học không giám sát, học bán giám sát và học tăng cường. Trong chuyên đề này, phương pháp học máy có giám sát được lựa chọn làm hướng tiếp cận chính, do bài toán thuộc nhóm phân loại nhị phân.

a) Học có giám sát (Supervised learning)

Trong học có giám sát, các thuật toán được huấn luyện trên những tập dữ liệu đã được gán nhãn, trong đó mỗi mẫu dữ liệu đều đi kèm với một nhãn mô tả kết quả hoặc lớp tương ứng. Nói cách khác, thuật toán không chỉ được cung cấp dữ liệu đầu vào mà còn được cung cấp một “khóa trả lời”, cho biết dữ liệu đó nên được diễn giải hoặc phân loại như thế nào.

Ví dụ, một thuật toán có thể được huấn luyện bằng tập ảnh hoa đã được gán nhãn loại hoa, từ đó giúp nó nhận diện đúng loại hoa khi được cung cấp một hình ảnh mới.

Học có giám sát thường được sử dụng để tạo ra các mô hình học máy phục vụ mục đích dự đoán và phân loại. Một số thuật toán học máy phổ biến dùng cho bài toán phân loại trong học máy có giám sát như: Logistic regression, Decision tree Support vector Machine, Random forest, XGBoost.



Hình 1.2. Mô hình học máy có giám sát

b) Học không giám sát (Unsupervised learning)

Học máy không giám sát sử dụng các tập dữ liệu chưa được gán nhãn để huấn luyện thuật toán. Trong quá trình này, thuật toán được cung cấp dữ liệu không có nhãn, buộc nó phải tự khám phá các mẫu và cấu trúc tiềm ẩn mà không có bất kỳ hướng dẫn nào từ bên ngoài.

Ví dụ: một thuật toán có thể được cung cấp một lượng lớn dữ liệu người dùng chưa gán nhãn thu thập từ một nền tảng mạng xã hội để xác định các xu hướng hành vi trên nền tảng đó.

Học máy không giám sát thường được các nhà nghiên cứu và nhà khoa học dữ liệu sử dụng để nhanh chóng và hiệu quả trong việc tìm kiếm các mẫu ẩn trong những tập dữ liệu lớn và chưa được gán nhãn.

c) Học bán giám sát (Semi-supervised learning)

Học máy bán giám sát sử dụng cả dữ liệu có nhãn và dữ liệu không có nhãn để huấn luyện thuật toán. Nhìn chung, trong quá trình học bán giám sát, thuật toán đầu tiên được cung cấp một lượng nhỏ dữ liệu có nhãn để giúp định hướng quá trình phát triển, sau đó được cung cấp một lượng lớn dữ liệu không có nhãn hơn để hoàn thiện mô hình.

Ví dụ, một thuật toán có thể được cung cấp một lượng nhỏ dữ liệu giọng nói có nhãn, sau đó được huấn luyện trên một tập dữ liệu giọng nói không có nhãn lớn hơn nhiều để tạo ra một mô hình có khả năng nhận dạng giọng nói.

Học bán giám sát thường được sử dụng để đào tạo các thuật toán cho mục đích phân loại và dự đoán khi không có sẵn khối lượng lớn dữ liệu được gán nhãn.

d) Học tăng cường (Reinforcement learning)

Học tăng cường sử dụng phương pháp thử và sai để huấn luyện thuật toán và xây dựng mô hình. Trong quá trình huấn luyện, thuật toán hoạt động trong các môi trường cụ thể và sau đó được cung cấp phản hồi sau mỗi kết quả. Giống như cách trẻ em học tập, thuật toán dần dần bắt đầu hiểu được môi trường của mình và bắt đầu tối ưu hóa các hành động để đạt được các kết quả cụ thể.

Ví dụ, một thuật toán có thể được tối ưu hóa bằng cách chơi cờ vua liên tiếp, cho phép nó học hỏi từ những thành công và thất bại trong quá khứ của mỗi ván cờ.

Học tăng cường thường được sử dụng để xây dựng các thuật toán phải đưa ra chuỗi quyết định hoặc hành động hiệu quả để đạt được mục tiêu, chẳng hạn như chơi trò chơi hoặc tóm tắt toàn bộ văn bản.

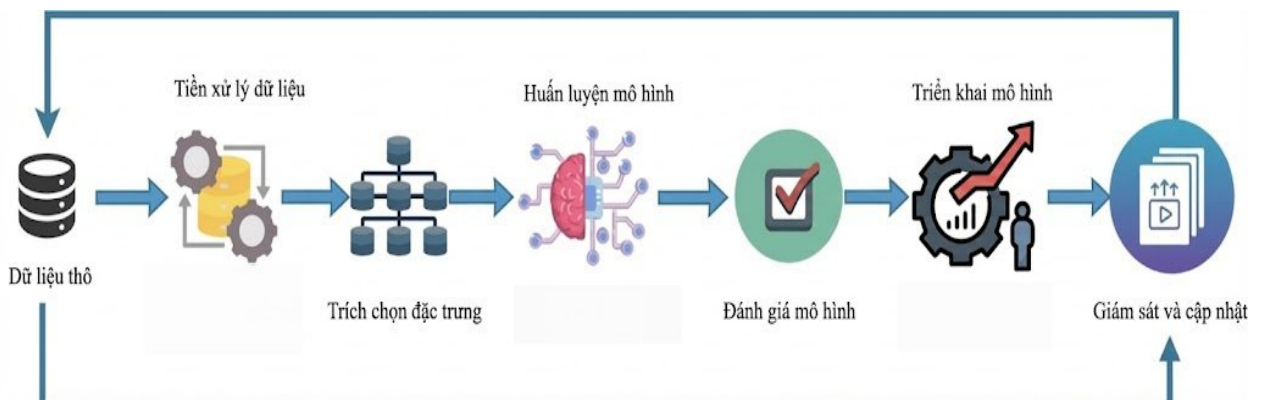
1.2.3. Quy trình cơ bản của học máy

Quy trình cơ bản của học máy thường bao gồm một chuỗi các bước từ chuẩn bị dữ liệu đến triển khai và giám sát mô hình. Các bước chính trong quy trình này được mô tả tổng quan như sau:

- **Dữ liệu thô (Raw Data):** Dữ liệu thô được áp dụng cho Chuyên đề chủ yếu là các URL của trang web, nội dung HTML. Dữ liệu được thu thập từ các nguồn uy tín như cơ sở dữ liệu PhishTank cho trang web lừa đảo, và các trang web hợp pháp từ các nguồn đáng tin cậy. Dữ liệu thô thường có tính đa dạng và phức tạp, đồng thời chứa nhiều thông tin chưa có cấu trúc rõ ràng.
- **Tiền xử lý dữ liệu (Data Preprocessing):** Dữ liệu thu thập được sẽ được làm sạch và chuẩn hóa. Cụ thể bao gồm việc loại bỏ các URL bị lỗi hoặc không truy cập được, chuẩn hóa dạng URL, loại bỏ dữ liệu nhiễu hoặc không đầy đủ, xử lý các giá trị thiếu.
- **Trích chọn đặc trưng (Feature Extraction):** Từ dữ liệu đã tiền xử lý, ta xây dựng các bộ đặc trưng như đặc trưng dựa trên URL (URL based feature), đặc trưng dựa trên siêu liên kết (hyperlink based feature). Việc kết hợp nhiều loại đặc trưng giúp mô hình học máy có thể nhận diện chính xác hơn các dấu hiệu của trang web lừa đảo so với chỉ dùng một loại đặc trưng đơn lẻ.
- **Huấn luyện mô hình (Model Training):** Sau khi có bộ đặc trưng hoàn chỉnh, các thuật toán học máy như Logistic regression, Decision

tree Support vector Machine, Random forest, XGBoost sẽ được sử dụng để huấn luyện mô hình phát hiện. Quá trình huấn luyện bao gồm việc lựa chọn thuật toán phù hợp, điều chỉnh tham số, và sử dụng các kỹ thuật như cross-validation để đảm bảo mô hình không bị quá khớp (overfitting).

- **Đánh giá mô hình (Model Evaluation):** Mô hình được đánh giá bằng các chỉ số như độ chính xác (accuracy), tỷ lệ phát hiện đúng (recall), độ chính xác dự đoán (precision), và F1-score. Các chỉ số này giúp đánh giá khả năng mô hình phân biệt chính xác giữa trang web lừa đảo và trang web hợp pháp trên tập dữ liệu kiểm thử riêng biệt. Việc đánh giá kỹ càng giúp đảm bảo tính tin cậy khi áp dụng mô hình vào thực tế.
- **Triển khai mô hình (Model Deployment):** Mô hình được triển khai vào các hệ thống phát hiện phishing thực tế, có thể tích hợp vào trình duyệt, phần mềm bảo mật hoặc hệ thống giám sát mạng để tự động phát hiện các trang web lừa đảo trong thời gian thực, giúp người dùng tránh bị lừa đảo và giảm thiểu rủi ro bảo mật.
- **Giám sát và cập nhật (Monitoring and Updating):** Do đặc điểm thay đổi liên tục của các trang web lừa đảo (thay đổi kỹ thuật, tạo URL mới, cập nhật nội dung giả mạo), mô hình cần được giám sát và cập nhật thường xuyên. Việc tái huấn luyện dựa trên dữ liệu mới giúp mô hình thích nghi với các xu hướng tấn công mới và duy trì hiệu quả phát hiện lâu dài.



Hình 1.3. Tổng quan về quy trình học máy

1.3. Kết luận chương 1

Chương 1 đã trình bày tổng quan về trang web lừa đảo, các phương pháp phát hiện phishing hiện nay cũng như những kiến thức nền tảng về kỹ thuật học máy. Qua phân tích có thể thấy rằng các phương pháp truyền thống như blacklist hoặc so sánh giao diện tồn tại nhiều hạn chế, đặc biệt trong việc phát hiện các cuộc tấn công zero-day. Trong khi đó, các phương pháp dựa trên học máy cho thấy tiềm năng vượt trội nhờ khả năng học từ dữ liệu và suy luận từ đặc trưng. Đây chính là cơ sở lý thuyết quan trọng để chuyên đề tập trung nghiên cứu mô hình phát hiện trang web lừa đảo dựa trên đặc trưng lai bằng kỹ thuật học máy trong các chương tiếp theo.

CHƯƠNG 2. MÔ HÌNH PHÁT HIỆN TRANG WEB LỪA ĐẢO DỰA TRÊN ĐẶC TRƯNG LAI BẰNG KỸ THUẬT HỌC MÁY

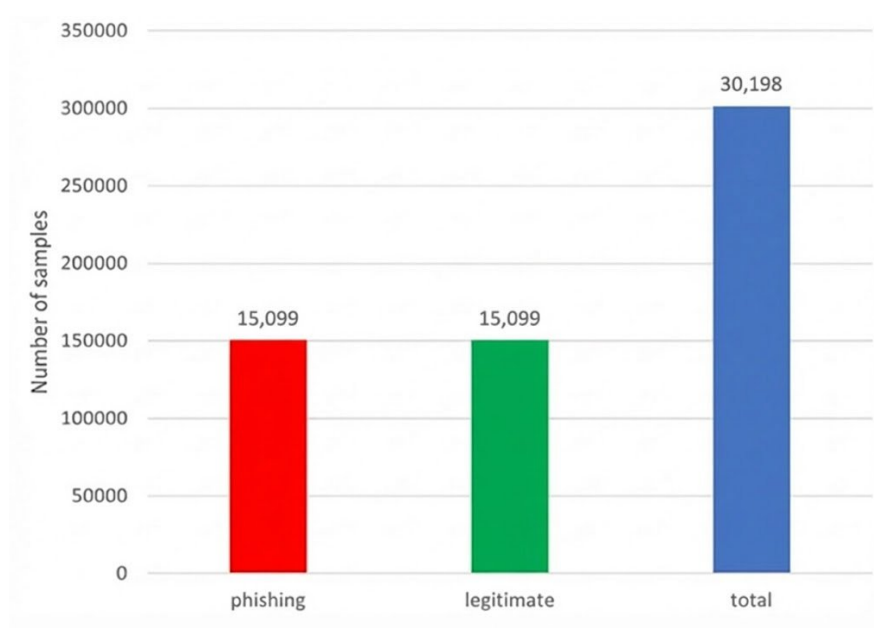
2.1. Xây dựng bộ dữ liệu

Tập dữ liệu được sử dụng để trích xuất các đặc trưng mong muốn được thu thập từ 30.198 trang web khác nhau, bao gồm các trang web lừa đảo (phishing) và hợp pháp (legitimate). Các trang web lừa đảo được thu thập khi chúng còn đang hoạt động, do vòng đời tồn tại của chúng thường rất ngắn.

Các URL lừa đảo được thu thập từ PhishTank, đây là một trung tâm hợp tác thu thập dữ liệu và thông tin về các vụ lừa đảo trực tuyến (phishing) trên Internet. Ngoài ra, PhishTank cung cấp API mở cho các nhà phát triển và nhà nghiên cứu để tích hợp dữ liệu chống lừa đảo vào ứng dụng của họ hoàn toàn miễn phí. Dữ liệu từ PhishTank được cộng đồng xác thực và thường xuyên cập nhật, do đó đảm bảo tính tin cậy cho quá trình huấn luyện và đánh giá mô hình.

Các URL hợp pháp được thu thập từ tập dữ liệu công khai từ Mendeley Data, bao gồm các trang web đáng tin cậy được thu thập từ cơ sở dữ liệu URLHaus. Tập dữ liệu này chứa một số lượng lớn các URL hợp pháp và được sử dụng rộng rãi trong các nghiên cứu về phát hiện phishing, giúp đảm bảo tính đại diện và khả năng tổng quát hóa của mô hình. Từ tập dữ liệu này, 15.099 URL hợp pháp được lựa chọn ngẫu nhiên để sử dụng trong thí nghiệm.

Các giá trị nhãn được gán là 0 cho URL hợp pháp (legitimate) và 1 cho URL lừa đảo (phishing).



Hình 2.1. Tỷ lệ tập dữ liệu được sử dụng

2.2. Lựa chọn đặc trưng

Độ chính xác của hệ thống phát hiện trang web lừa đảo phụ thuộc vào các đặc trưng để phân biệt giữa các URL lừa đảo và lành tính. Các đặc trưng mà chuyên đề áp dụng được dựa trên URL và siêu liên kết (hyperlink) của trang web.

Các đặc trưng dựa trên URL được trích xuất thông qua việc phân tích cấu trúc URL, trong khi các đặc trưng dựa trên hyperlink được lấy từ mã nguồn của trang web bằng cách tạo Mô hình Đối tượng Tài liệu (Document Object Model – DOM). Nhờ cấu trúc dạng cây có tổ chức của các tài liệu XML hoặc HTML, DOM giúp việc tìm kiếm và phân tích các thông tin liên quan đến hyperlink trở nên dễ dàng hơn.

Trước hết, các luật trích xuất đặc trưng được xây dựng, đây là bước rất quan trọng nhằm phát hiện các mẫu ẩn và mối quan hệ trong tập dữ liệu. Do đó, cấu trúc URL và cây DOM được phân tích kỹ lưỡng để xây dựng các luật đáng tin cậy, và các đặc trưng được trích xuất dựa trên những luật này. Các đặc trưng thu được được kết hợp lại với nhau để tạo thành một tập đặc trưng hỗn hợp, dùng để huấn luyện mô hình phân loại trang web bằng nhiều thuật toán máy học khác nhau.

Bảng 2.1. Các đặc trưng phân loại được áp dụng

ID	Loại	Tên đặc trưng	Tổng số đặc trưng
1	Các đặc trưng dựa trên URL (URL based features)	Tên miền của URL Đếm số lượng tên miền phụ trong URL Địa chỉ IP trong tên miền Biểu tượng “@” trong URL Độ dài của URL Độ sâu của URL Chuyển hướng “//” trong URL “http/https” trong tên miền https trong scheme Sử dụng dịch vụ rút gọn URL “TinyURL” Tiền tố hoặc hậu tố “-” trong tên miền Sự tồn tại của từ nhạy cảm Sự tồn tại của tên thương hiệu phổ biến Sự tồn tại của chữ cái in hoa Số lượng dấu chấm trong URL	15

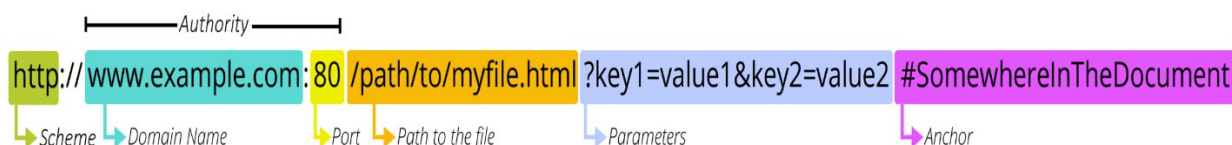
2	Các đặc trưng dựa trên siêu liên kết (Hyperlink based features)	Không có siêu liên kết Tỷ lệ siêu liên kết nội bộ Tỷ lệ siêu liên kết bên ngoài CSS nội bộ/ bên ngoài Hành động liên kết biểu mẫu đáng ngờ Siêu liên kết null Favicon nội bộ/ bên ngoài Tỷ lệ phát hiện trang chung Tỷ lệ trang chung trong phần chân trang Trình xử lý biểu mẫu phía máy chủ	10
---	---	--	----

2.2.1. Đặc trưng dựa trên URL

Hệ thống định vị tài nguyên thống nhất (Uniform Resource Locator – URL) được sử dụng để xác định vị trí của các tài nguyên trên Internet như hình ảnh, tệp âm thanh hoặc video, các trang siêu văn bản,... Hình 2.2 minh họa cấu trúc của một URL, trong đó URL được chia thành nhiều thành phần khác nhau.

Một URL thường bắt đầu bằng giao thức (scheme) như HTTP, HTTPS, FTP,... dùng để xác định phương thức truy cập tài nguyên; trong đó HTTPS (Hypertext Transfer Protocol Secure) là giao thức an toàn, được sử dụng phổ biến hiện nay. Tiếp theo là thành phần định danh máy chủ (authority), bao gồm tên miền (domain name) hoặc địa chỉ IP, và có thể kèm theo cổng (port), nhằm xác định máy chủ lưu trữ tài nguyên; trong đó tên miền (domain name) có cấu trúc phân cấp gồm subdomain, tên miền chính (Second-Level Domain - SLD) và miền cấp cao nhất (Top Level Domain - TLD). Trong hình 2.2, “www” là subdomain, “example” là tên miền chính và “.com” là miền cấp cao nhất.

Sau thành phần authority là đường dẫn (path), chỉ ra vị trí cụ thể của tài nguyên trên máy chủ; tham số truy vấn (query), dùng để truyền dữ liệu giữa trình duyệt và máy chủ; và cuối cùng là mảnh (fragment), dùng để định vị đến một phần cụ thể trong tài liệu.



Hình 2.2. Các thành phần của một URL

Kẻ lừa đảo có toàn quyền kiểm soát tên miền (domain name), tên miền phụ (subdomain) và đường dẫn (path) trong URL. Những đặc trưng được áp dụng dưới đây sẽ trình bày cách mà tội phạm mạng sử dụng kỹ thuật làm rối cấu trúc URL để đánh lừa người dùng.

- **Tên miền của URL (Domain name of URL):** Đặc trưng này lưu trữ toàn bộ tên miền (domain name) của URL. Tuy nhiên, đặc trưng này không mang lại hiệu quả đáng kể trong giai đoạn triển khai và vì vậy được loại bỏ trong quá trình huấn luyện mô hình.

$$UF1 = \text{Tên miền của URL}$$

- **Đếm số lượng tên miền phụ trong URL (Count subdomain in URL):** Đặc trưng này kiểm tra số lượng dấu chấm trong phần tên miền (domain name) của URL. Thông thường, một URL hợp pháp chỉ có hai dấu chấm trong tên miền, không tính tiền tố “www.”. Tuy nhiên, trong các URL lừa đảo, kẻ tấn công thường tạo thêm nhiều tên miền phụ (subdomain), làm số dấu chấm tăng lên nhằm đánh lừa người dùng. Nếu tên miền có 3 dấu chấm, URL được xem là đáng ngờ; nếu tên miền có hơn 3 dấu chấm, URL được phân loại là lừa đảo

$$UF2 = \begin{cases} 1, \text{nếu số dấu chấm trong tên miền} > 3 \text{ (lừa đảo)} \\ 0.5, \text{nếu số dấu chấm trong tên miền} = 3 \text{ (đáng ngờ)} \\ 0, \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Địa chỉ IP là tên miền (IP Address is Domain Name):** Trong một số trường hợp, kẻ tấn công sử dụng địa chỉ IP thay cho tên miền (domain name) của URL. Khi URL chứa địa chỉ IP (IPv4 hoặc IPv6) ở phần tên miền, điều này cho thấy trang web có khả năng đang cố gắng đánh cắp thông tin cá nhân của người dùng.

$$UF3 = \begin{cases} 1, \text{nếu tên miền là địa chỉ IP (lừa đảo)} \\ 0, \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Biểu tượng “@” trong URL (“@” symbol in URL):** Đặc trưng này kiểm tra sự xuất hiện của ký tự “@” trong URL. Thông thường, kẻ tấn công chèn ký tự “@” vào sau tên miền (domain name) của trang web hợp pháp để đánh lừa người dùng rằng đó là một URL đáng tin cậy. Khi người dùng nhấp vào liên kết này, trình duyệt sẽ bỏ qua toàn bộ phần trước ký tự “@” và truy cập vào địa chỉ thực nằm sau ký tự này.

$$UF4 = \begin{cases} 1, \text{nếu URL chứa ký tự "@" (lừa đảo)} \\ 0, \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Độ dài của URL (Length of URL):** Kẻ tấn công thường tạo ra các URL lừa đảo có độ dài lớn nhằm che giấu những thành phần đáng ngờ trong URL và đánh cắp dữ liệu người dùng. Khi số lượng ký tự trong URL tăng lên, khả năng URL đó là lừa đảo cũng tăng theo. Trên thực tế, không tồn tại một ngưỡng độ dài cố định để phân biệt rõ ràng giữa URL hợp pháp và URL lừa đảo. Một số nghiên cứu chỉ ra rằng, độ dài của các URL hợp pháp trung bình là 75 ký tự. Nếu độ dài URL lớn hơn 75 nhưng nhỏ hơn 100, URL được xem là đáng ngờ và giá trị đặc trưng được gán là 0.5.

$$UF5 = \begin{cases} 0, & \text{nếu độ dài của URL} < 75 \text{ (hợp pháp)} \\ 0.5, & \text{nếu độ dài của URL} < 100 \text{ (đáng ngờ)} \\ 1, & \text{trường hợp còn lại (lừa đảo)} \end{cases}$$

- **Độ sâu của URL (Depth of URL):** Đặc trưng này tính toán số lượng trang con trong URL của một trang web. Các trang con được phân tách bằng dấu gạch chéo (“/”) trong phần đường dẫn (path) của URL. Thông thường, cấu trúc thư mục của các trang web chính thống không quá sâu. Ngược lại, các kẻ tấn công thường lưu trữ các trang giả mạo ở những cấp thư mục rất sâu trên máy chủ. Do đó, đường dẫn tệp của các trang web lừa đảo thường dài hơn so với trang web hợp pháp. Giá trị của đặc trưng này là một giá trị số, được tính toán trực tiếp từ URL.

$$UF6 = \text{Tổng số trang con, thư mục con trong đường dẫn URL}$$

- **Chuyển hướng “//” trong URL (Redirection “//” in URL):** Ký hiệu “//” được sử dụng để chuyển hướng người dùng sang các trang web khác. Các đối tượng lừa đảo thường lợi dụng ký hiệu này trong URL nhằm chuyển hướng người dùng đến các trang web giả mạo. Trong các URL hợp lệ, ký hiệu “//” thường xuất hiện ở vị trí thứ 6 hoặc 7 (sau “http:” hoặc “https:”). Do đó, nếu vị trí xuất hiện của “//” lớn hơn 7, có thể kết luận rằng ký hiệu “//” xuất hiện ở vị trí bất thường, không tuân theo quy tắc chuẩn.

$$UF7 = \begin{cases} 1, & \text{nếu "//" không ngay sau http/https (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **“http/https” trong tên miền (“http/https” in domain name):** Kẻ lừa đảo có thể chèn chuỗi “http” hoặc “https” vào phần tên miền (domain name) của URL nhằm gây nhầm lẫn cho người dùng. Đặc trưng này phát hiện sự xuất hiện của “http/https” trong tên miền. Nếu tên miền

chứa “http/https”, giá trị của đặc trưng được gán là 1 (lừa đảo), ngược lại là 0 (hợp pháp)

$$UF8 = \begin{cases} 1, & \text{nếu http/https tồn tại trong domain (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **https trong scheme (https in scheme):** Đặc trưng này kiểm tra giao thức của URL. Nếu giao thức của URL là “https” thì giá trị đặc trưng được gán là 0 (hợp pháp), ngược lại là 1 (lừa đảo). Giao thức là một thành phần rất quan trọng của URL. Phần lớn các trang web hợp pháp sử dụng giao thức “https” để đảm bảo kết nối an toàn khi truyền tải thông tin nhạy cảm. Tuy nhiên, hiện nay các đối tượng lừa đảo cũng sử dụng các kết nối “https” giả mạo để đánh lừa người dùng. Do đó, đặc trưng này không còn mang lại hiệu quả cao trong việc phân biệt trang web lừa đảo và trang web hợp pháp.

$$UF9 = \begin{cases} 1, & \text{nếu giao thức URL là http (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Sử dụng dịch vụ rút gọn URL “TinyURL” (Using URL shortening service “tinyURL”):** Trên World Wide Web, rút gọn URL là một kỹ thuật giúp làm ngắn địa chỉ URL trong khi vẫn liên kết tới trang web đích thông qua cơ chế chuyển hướng. Ví dụ, URL “https://w.wiki/U” là dạng rút gọn của “https://en.wikipedia.org/wiki/URL_shortening”. Tuy nhiên, hiện nay nhiều kẻ lừa đảo lợi dụng các dịch vụ rút gọn URL để đánh lừa người dùng. Do đó, trong đặc trưng này, chúng tôi kiểm tra xem URL có sử dụng dịch vụ rút gọn URL hay không.

$$UF10 = \begin{cases} 1, & \text{nếu URL rút gọn tồn tại (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Tiền tố hoặc hậu tố “-” trong tên miền (Prefix or Suffix “-” in domain name):** Đặc trưng này kiểm tra sự xuất hiện của “-” trong phần tên miền (domain name) của URL. Trong các tên miền hợp pháp, dấu gạch ngang hiếm khi được sử dụng. Để đánh lừa nạn nhân, kẻ tấn công thường thêm các tiền tố hoặc hậu tố được phân tách bằng dấu “-” vào tên miền. Điều này khiến địa chỉ trông giống như trang web hợp pháp, từ đó dụ người dùng truy cập vào trang web lừa đảo.

$$UF11 = \begin{cases} 1, & \text{nếu phần tên miền có chứa “-” (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Sự tồn tại của từ nhạy cảm (Existence of sensitive word):** Các URL lừa đảo thường chứa những từ hoặc token phổ biến như “login”, “update”, “validate”, “activate”, “secure”,... được kẻ sử dụng trong URL nhằm tạo cảm giác khẩn cấp và thúc đẩy người dùng truy cập vào trang web lừa đảo để nhanh chóng đánh cắp thông tin nhạy cảm. Một danh sách thuật ngữ lừa đảo được lập lên từ những tài liệu tham khảo. Vì vậy, URL chứa bất kỳ thuật ngữ nào trong danh sách, giá trị đặc trưng được gán là 1 (lừa đảo), ngược lại là 0 (hợp pháp).

$$UF12 = \begin{cases} 1, & \text{nếu URL có chứa bất kỳ từ nhạy cảm nào (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Sự tồn tại của tên thương hiệu phổ biến (Existence of trendy brand name):** Các trang web lừa đảo thường được tạo ra bằng cách giả mạo các trang web của những thương hiệu nổi tiếng. Do đó, kẻ tấn công thường chèn tên các thương hiệu đang được quan tâm vào URL của các trang phishing nhằm đánh lừa người dùng. Khi nhìn thấy tên thương hiệu xuất hiện trong URL, người dùng có thể nhầm tưởng đó là trang web chính thức của thương hiệu. Trong chuyên đề này sẽ sử dụng một danh sách gồm 19 thương hiệu phổ biến thường bị kẻ tấn công lợi dụng.

$$UF13 = \begin{cases} 1, & \text{nếu URL có chứa bất kỳ tên thương hiệu phổ biến nào (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Sự tồn tại của chữ cái in hoa (Existence of upper case letter):** Các URL hợp pháp thường được viết bằng chữ thường. Nhưng các URL lừa đảo thường chứa chữ cái in hoa nhằm đánh lừa người dùng. Nếu URL chứa bất kỳ chữ cái in hoa nào thì được coi là lừa đảo.

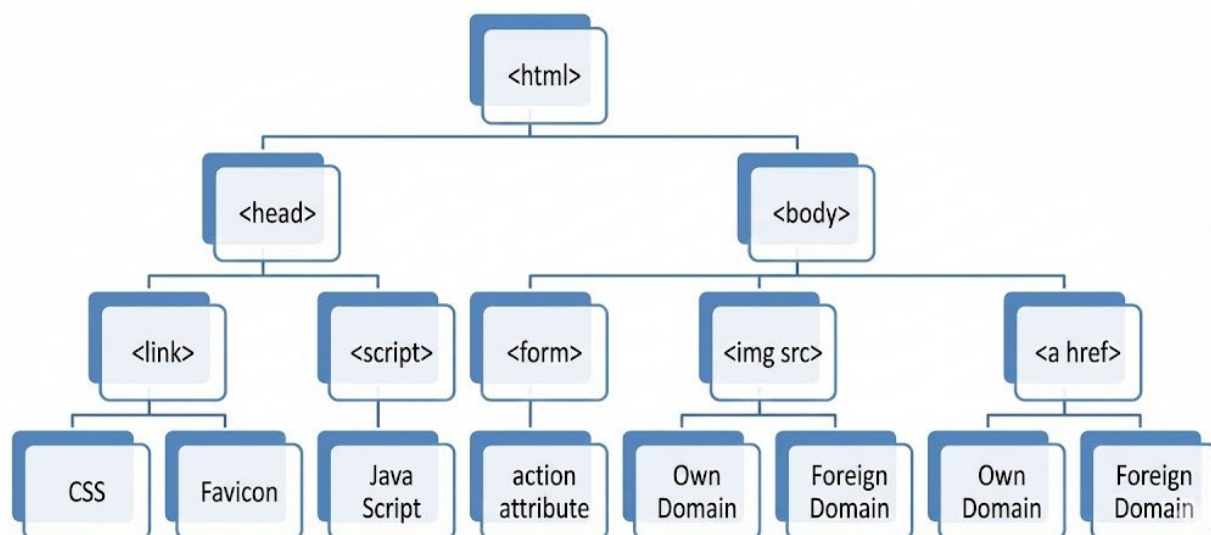
$$UF14 = \begin{cases} 1, & \text{nếu URL có chứa bất kỳ chữ cái in hoa nào (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Số lượng dấu chấm trong URL (Number of dots in URL):** URL của các trang web hợp pháp thường không có quá hai dấu chấm không tính tiền tố “www.”. Do đó, đặc trưng này tính tổng số dấu chấm trong toàn bộ URL. Nếu số dấu chấm nhiều hơn 2, thì URL được phân loại là lừa đảo, ngược lại thì là hợp pháp.

$$UF15 = \begin{cases} 1, & \text{nếu số chấm} > 2 \text{ (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

2.2.2. Đặc trưng dựa trên siêu liên kết

Trong phần này, các đặc trưng siêu liên kết sẽ được trích xuất từ mã nguồn của trang web. Thông tin về các siêu liên kết được xem xét thông qua mô hình cây Document Object Model (DOM), đây là phương pháp biểu diễn một trang web theo cấu trúc phân cấp có tổ chức, giúp việc duyệt và phân tích tài liệu trở nên dễ dàng hơn.



Hình 2.3. Cây DOM HTML

Mô hình DOM cho phép truy cập và thao tác thuận tiện với các thành phần của trang web như thẻ (tags), định danh (IDs), lớp (classes), thuộc tính (attributes) và các phần tử (elements). Trong quá trình trích xuất đặc trưng, chúng tôi chủ yếu tập trung vào các thẻ link, form, src và anchor tags. Hình 2.3 minh họa một cây DOM điển hình. Tổng cộng 10 đặc trưng dựa trên siêu liên kết đã được trích xuất.

- **Không có siêu liên kết (No Hyperlink):** Các trang web hợp pháp thường bao gồm nhiều trang web khác nhau và do đó chứa nhiều siêu liên kết. Ngược lại, các trang web lừa đảo thường chỉ có số lượng trang rất hạn chế. Ngoài ra, trong trường hợp kẻ tấn công sử dụng chiến lược ẩn siêu liên kết, trang web lừa đảo có thể không cung cấp bất kỳ siêu liên kết nào. Một trang web xác thực thường có ít nhất một siêu liên kết tồn tại trong mã nguồn của trang. Tổng số siêu liên kết được xác định bằng cách đếm các thẻ href, link và src trong mã nguồn HTML.

$$HF1 = \begin{cases} 0, & \text{tổng số siêu liên kết} > 0 \text{ (hợp pháp)} \\ 1, & \text{tổng số siêu liên kết} = 0 \text{ (lừa đảo)} \end{cases}$$

- **Tỷ lệ siêu liên kết nội bộ (Internal Hyperlink ratio):** Siêu liên kết nội bộ là các liên kết trỏ tới cùng tên miền gốc của trang web. Kẻ tấn công thường sao chép mã nguồn của trang web chính thức để tạo trang lừa đảo giống thật, do đó nhiều liên kết trong mã nguồn cũng trỏ về trang web giả mạo. Trên trang web hợp pháp, hầu hết liên kết đều cùng tên miền cơ sở, trong khi trang lừa đảo thường có nhiều liên kết trỏ đến tên miền chính thức bị giả mạo. Đặc trưng này tính tỷ lệ liên kết nội bộ trên tổng số liên kết trong mã nguồn trang web.

$$\text{Tỷ lệ liên kết nội bộ} = \begin{cases} \frac{\text{Tổng liên kết nội bộ}}{\text{Tổng siêu liên kết}}, & \text{nếu tổng siêu liên kết} > 0 \\ 0, & \text{nếu tổng siêu liên kết} = 0 \end{cases}$$

$$HF2 = \begin{cases} 0, & \text{nếu tỷ lệ liên kết nội bộ} \geq 0.5 \text{ (hợp pháp)} \\ 1, & \text{trường hợp còn lại (lừa đảo)} \end{cases}$$

- **Tỷ lệ siêu liên kết bên ngoài (External Hyperlink ratio):** Siêu liên kết bên ngoài là các liên kết trỏ đến tên miền khác. Hầu hết các liên kết trên trang web lừa đảo là liên kết bên ngoài, trong khi đó trang web chính thức thường chỉ sử dụng một số ít. Đặc trưng này tính tỷ lệ liên kết bên ngoài trên tổng số liên kết trong mã nguồn của trang web.

$$\begin{aligned} & \text{Tỷ lệ liên kết} \\ & \text{bên ngoài} \\ & = \begin{cases} \frac{\text{Tổng liên kết ngoài}}{\text{Tổng siêu liên kết}}, & \text{nếu tổng siêu liên kết} > 0 \\ 0, & \text{nếu tổng siêu liên kết} = 0 \end{cases} \\ HF3 & = \begin{cases} 1, & \text{nếu tỷ lệ liên kết ngoài} > 0.5 \text{ (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases} \end{aligned}$$

- **CSS nội bộ/ bên ngoài (Internal/ External CSS):** CSS (Cascading Style Sheets) là ngôn ngữ định dạng giao diện cho các trang web HTML, XHTML và XML. Các trang lừa đảo thường giả mạo giao diện trang chính thức để đánh lừa người dùng. Thay vì tự tạo CSS, kẻ tấn công thường dùng lại file CSS bên ngoài của trang chính thức. CSS gồm hai loại: nội bộ (nhúng trực tiếp trong mã HTML) và bên ngoài (liên kết qua thẻ <link> với thuộc tính rel="stylesheet" và href="url"). Đặc trưng này kiểm tra sự tồn tại của file CSS bên ngoài trong mã nguồn trang web.

$$HF4 = \begin{cases} 1, \text{ nếu file CSS bên ngoài có} \\ \text{tên miền} \neq \text{tên miền gốc (lừa đảo)} \\ 0, \text{ trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Hành động liên kết biểu mẫu đáng ngờ (Suspicious Form Link Action):** Các trang web lừa đảo thường có biểu mẫu đăng nhập hoặc đăng ký nhằm chiếm đoạt thông tin của nạn nhân. Khi người dùng nhập thông tin trên trang giả mạo, toàn bộ dữ liệu sẽ bị chuyển tới kẻ tấn công. Thông thường, thuộc tính action của thẻ <form> chứa URL của chính trang web thật. Tuy nhiên, biểu mẫu đăng nhập giả thường có trường action chứa liên kết ngoài hoặc file PHP. Đôi khi, trường action có giá trị rỗng hoặc chứa các giá trị như 'null', '#', 'javascript:void()',... Do đó, đặc trưng này kiểm tra giá trị của trường action trong thẻ <form> để xác định tính xác thực của biểu mẫu đăng nhập.

$$HF5 = \begin{cases} 1, \text{ nếu liên kết ngoài hoặc file PHP hoặc "" hoặc} \\ \text{"javascript:void()" trong trường action (lừa đảo)} \\ 0, \text{ trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Siêu liên kết null (Null Hyperlink):** Đặc trưng này chỉ xét thẻ <a>. Nó tính tỷ lệ phần trăm các liên kết null trên tổng số liên kết <a> trên trang web. Kế hoạch của kẻ tấn công là giữ người dùng trên cùng một trang cho đến khi họ nhập thông tin nhạy cảm. Do đó, khi người dùng nhấp vào bất kỳ liên kết nào trên trang đăng nhập, họ đều bị dẫn trở lại chính trang đó. Điều này được thực hiện thông qua các liên kết có dạng: ,

$$HF6 = \begin{cases} \frac{\text{Tổng số liên kết null}}{\text{Tổng số liên kết } \langle a \rangle}, \text{ nếu tổng số liên kết } \langle a \rangle > 0 \\ 0, \text{ nếu tổng số liên kết } \langle a \rangle = 0 \end{cases}$$

$$HF6 = \begin{cases} 1, \text{ nếu tỷ lệ liên kết null} > 0.34 \text{ (lừa đảo)} \\ 0, \text{ trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Favicon nội bộ/ bên ngoài (Internal/External Favicon):** Favicon là một biểu tượng nhỏ đại diện cho nhận diện của trang web. Favicon được thêm vào trang web thông qua thẻ <link>. Nếu favicon hiển thị

trên thanh địa chỉ thuộc về một tên miền bên ngoài, trang web đó được coi là trang lừa đảo. Lý do là kẻ tấn công thường sao chép favicon của trang chính thức, khiến nhiều người dùng bị đánh lừa bởi thanh địa chỉ giả mạo với favicon trùng lặp. Do đó, thẻ <link> chứa favicon sẽ được phân tích và kiểm tra xem liên kết đó có cùng tên miền hay không.

$$HF7 = \begin{cases} 1, & \text{nếu favicon bên ngoài (lừa đảo)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

- **Tỷ lệ phát hiện trang chung (Common Page Detection ratio):** Kẻ tấn công thường bỏ ít thời gian để xây dựng nhiều trang web giả. Chúng nhanh chóng tạo các liên kết <a> để làm trang trông hợp pháp. Tuy nhiên, nhiều liên kết này lại dẫn đến cùng một trang chung duy nhất. Điều này khiến tỷ lệ phát hiện trang chung trên các trang lừa đảo cao hơn. Đặc trưng này tính tỷ lệ giữa số lần xuất hiện của liên kết phổ biến nhất trên tổng số liên kết <a> của trang web.

$$HF8 = \begin{cases} \frac{\text{tần suất của liên kết} \\ \langle a \rangle \text{ phổ biến nhất}}{\text{tổng số liên kết } \langle a \rangle}, & \text{nếu tổng số liên kết } \langle a \rangle > 0 \\ & \text{(lừa đảo)} \\ 0, & \text{nếu tổng số liên kết } \langle a \rangle = 0 \text{ (hợp pháp)} \end{cases}$$

- **Tỷ lệ trang chung trong phần chân trang (Common Page in Footer section ratio):** Đặc trưng này tương tự như đặc trưng trước, nhưng tập trung vào việc phát hiện các trang chung xuất hiện nhiều trong phần chân trang (footer) của trang web.

$$HF9 = \begin{cases} \frac{\text{tần suất liên kết phổ} \\ \text{biến nhất ở chân trang}}{\text{tổng số liên kết } \langle a \rangle \text{ ở chân trang}}, & \text{nếu tổng số liên kết } \langle a \rangle > 0 \\ & \text{(lừa đảo)} \\ 0, & \text{nếu tổng số liên kết } \langle a \rangle = 0 \text{ (hợp pháp)} \end{cases}$$

- **Trình xử lý biểu mẫu phía máy chủ (Server Form Handler (SFH)):** SFH chính là nơi mà dữ liệu người dùng nhập vào form sẽ được gửi tới. Nếu SFH chứa chuỗi rỗng hoặc giá trị “about:blank”, điều này có thể được xem là một hành vi lừa đảo nhằm đánh cắp thông tin nhạy cảm, bởi vì dữ liệu người dùng nhập vào biểu mẫu cần được gửi đến máy chủ để xử lý. Ngoài ra, nếu tên miền của SFH là

một domain bên ngoài, điều này cho thấy trang web có dấu hiệu đáng ngờ.

$$HF10 = \begin{cases} 1, & \text{nếu SFH chứa "" hoặc "about:blank" (lừa đảo)} \\ 0.5, & \text{nếu tên miền SFH là domain ngoài (đáng ngờ)} \\ 0, & \text{trường hợp còn lại (hợp pháp)} \end{cases}$$

2.2.3. Đặc trưng lại

Để nâng cao độ chính xác, hai nhóm đặc trưng gồm: đặc trưng dựa trên URL (URL based features) và đặc trưng dựa trên siêu liên kết (Hyperlink based features) được kết hợp nhằm xây dựng một tập đặc trưng lai (hybrid feature set). Tổng cộng, tập đặc trưng bao gồm 25 đặc trưng. Trong bước tiền xử lý, đặc trưng tên miền (UF1) được loại bỏ trước khi chia tập dữ liệu thành tập huấn luyện và tập kiểm thử, do đây không phải là đặc trưng số và không có ý nghĩa trong việc phân biệt trang web lừa đảo với trang web hợp pháp.

2.3. Mô hình phát hiện

Sau khi xây dựng tập đặc trưng lai bao gồm các đặc trưng dựa trên URL và siêu liên kết, bước tiếp theo là lựa chọn và xây dựng mô hình học máy phù hợp để thực hiện nhiệm vụ phân loại trang web. Trong chuyên đề này, thuật toán Extreme Gradient Boosting (XGBoost) được lựa chọn để xây dựng mô hình phát hiện trang web lừa đảo.

2.3.1. Lý do lựa chọn thuật toán XGBoost

XGBoost là một thuật toán học máy có giám sát, thuộc nhóm ensemble learning, dựa trên nguyên lý Gradient Boosting. Thuật toán này đã chứng minh được hiệu quả vượt trội trong nhiều bài toán phân loại và hồi quy, đặc biệt là các bài toán liên quan đến dữ liệu có số chiều lớn và các đặc trưng không tuyến tính.

Việc lựa chọn XGBoost cho bài toán phát hiện trang web lừa đảo trong chuyên đề này xuất phát từ các lý do sau:

- Khả năng mô hình hóa quan hệ phi tuyến mạnh mẽ: Các đặc trưng URL và hyperlink có mối quan hệ phức tạp, không tuyến tính. XGBoost sử dụng các cây quyết định làm bộ học cơ sở (base learner), giúp nắm bắt hiệu quả các mối quan hệ này.
- Khả năng xử lý tốt tập đặc trưng lai: Tập đặc trưng trong chuyên đề bao gồm cả đặc trưng nhị phân, đặc trưng số và đặc trưng tỷ lệ. XGBoost không yêu cầu chuẩn hóa dữ liệu nghiêm ngặt và hoạt động hiệu quả với các kiểu dữ liệu khác nhau.

- Giảm thiểu hiện tượng quá khớp (overfitting): Thuật toán tích hợp các kỹ thuật regularization (L1, L2), cắt tỉa cây (tree pruning) và shrinkage, giúp mô hình có khả năng tổng quát hóa tốt hơn.
- Hiệu năng cao và tốc độ huấn luyện nhanh: Nhờ cơ chế song song hóa và tối ưu hóa bộ nhớ, XGBoost cho phép huấn luyện mô hình nhanh ngay cả với tập dữ liệu có kích thước lớn.
- Được sử dụng rộng rãi trong các nghiên cứu phát hiện lừa đảo: Nhiều công trình nghiên cứu gần đây đã chỉ ra rằng XGBoost cho kết quả vượt trội trong bài toán phát hiện trang web lừa đảo.

2.3.2. Tổng quan thuật toán XGBoost

XGBoost (Extreme Gradient Boosting) là phiên bản mở rộng và tối ưu của thuật toán Gradient Boosting. Ý tưởng cốt lõi của Gradient Boosting là xây dựng một mô hình mạnh bằng cách kết hợp nhiều mô hình yếu (weak learners), trong đó mỗi mô hình mới được huấn luyện để khắc phục sai số của các mô hình trước đó.

Giả sử tập huấn luyện gồm n mẫu dữ liệu (x_i, y_i) , với x_i là vector đặc trưng và y_i là nhãn tương ứng. Mô hình XGBoost dự đoán đầu ra bằng cách cộng dồn kết quả của nhiều cây quyết định:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

trong đó f_k là cây quyết định thứ k và K là số lượng cây trong mô hình.

Hàm mục tiêu (objective function) của XGBoost bao gồm hai thành phần là hàm mất mát và hàm regularization:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

trong đó:

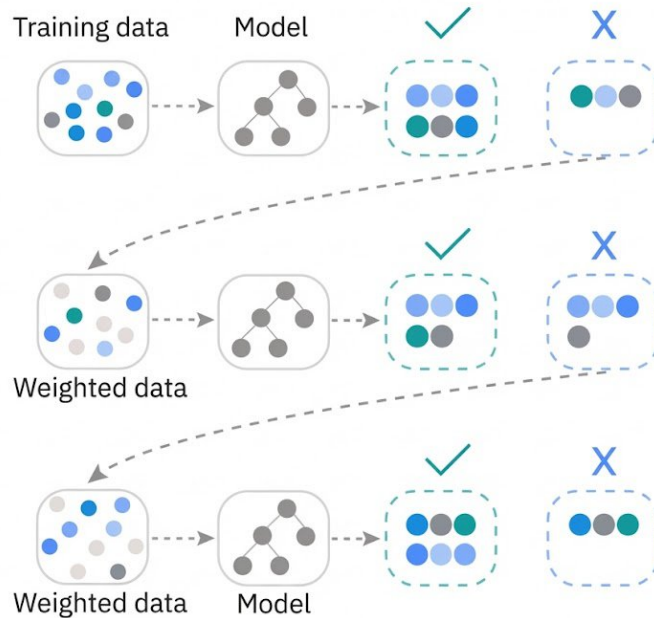
- $l(y_i, \hat{y}_i)$: là hàm mất mát (loss function), đo lường sự sai lệch giữa giá trị dự đoán và nhãn thực.
- $\Omega(f_k)$: là hàm regularization, dùng để kiểm soát độ phức tạp của cây quyết định, giúp hạn chế overfitting.

Hàm regularization thường được biểu diễn dưới dạng:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

trong đó:

- T là số lượng lá của cây.
- w_j là trọng số của lá thứ j .
- γ và λ là các hệ số regularization.



Hình 2.4. Minh họa mô hình XGBoost dựa trên tập hợp các cây quyết định

2.3.3. Quy trình xây dựng mô hình phát hiện

Quy trình xây dựng mô hình phát hiện trang web lừa đảo bằng XGBoost trong chuyên đề này được thực hiện theo các bước sau:

Bước 1: Chuẩn bị dữ liệu đầu vào

Sau khi trích xuất và kết hợp các đặc trưng URL-based và hyperlink-based, tập dữ liệu gồm 25 đặc trưng được hình thành. Đặc trưng tên miền (domain name) được loại bỏ do không phải là đặc trưng số và không mang lại hiệu quả phân loại. Các giá trị thiếu (nếu có) được xử lý trước khi đưa vào mô hình.

Bước 2: Chia tập dữ liệu

Tập dữ liệu được chia thành hai phần:

- Tập huấn luyện (training set): dùng để huấn luyện mô hình XGBoost.
- Tập kiểm thử (testing set): dùng để đánh giá hiệu năng của mô hình.

Tỷ lệ chia tập được lựa chọn sao cho đảm bảo mô hình có đủ dữ liệu để học và vẫn có tập kiểm thử độc lập để đánh giá khả năng tổng quát hóa.

Bước 3: Huấn luyện mô hình XGBoost

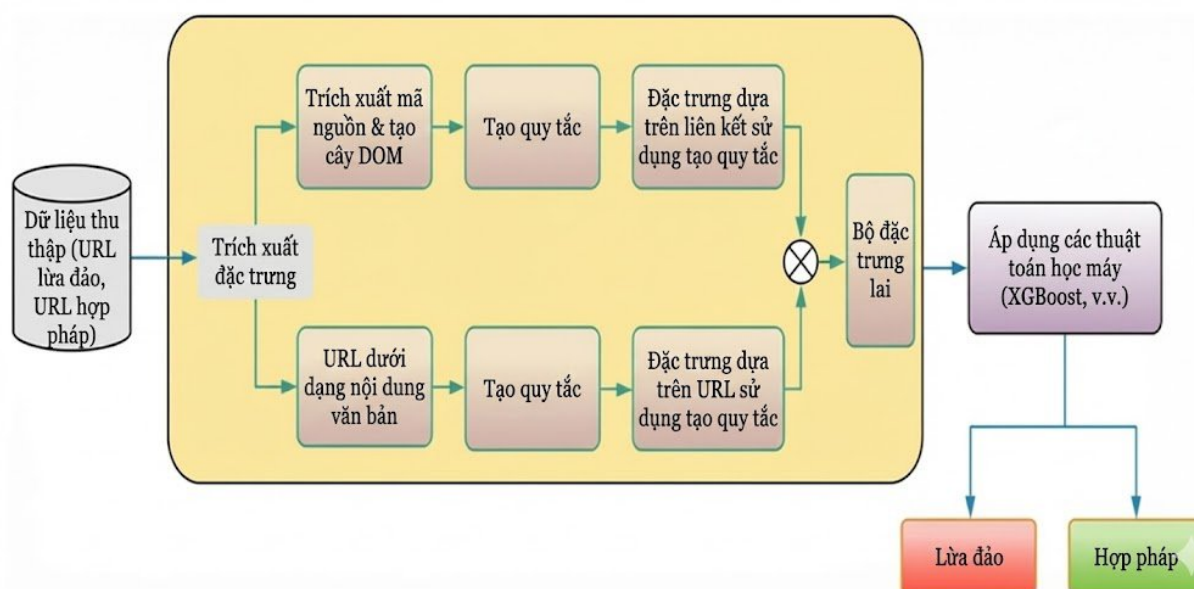
Mô hình XGBoost được huấn luyện trên tập huấn luyện với nhiệm vụ phân loại nhị phân, trong đó:

- Nhãn 0 tương ứng với trang web hợp pháp (legitimate).
- Nhãn 1 tương ứng với trang web lừa đảo (phishing).

Trong quá trình huấn luyện, các tham số quan trọng của XGBoost như số lượng cây (n_estimators), độ sâu tối đa của cây (max_depth), tốc độ học (learning_rate), và các hệ số regularization được điều chỉnh nhằm đạt được hiệu quả tốt nhất.

Bước 4: Dự đoán và phân loại

Sau khi huấn luyện, mô hình được sử dụng để dự đoán nhãn của các mẫu trong tập kiểm thử. Dựa trên kết quả dự đoán, các trang web được phân loại thành hai lớp: lừa đảo hoặc hợp pháp.



Hình 2.5. Mô tả quy trình của phương pháp được đề xuất

2.3.4. Ưu điểm của mô hình được đề xuất

Mô hình phát hiện trang web lừa đảo sử dụng XGBoost kết hợp tập đặc trưng lai mang lại nhiều ưu điểm:

- Khả năng phát hiện hiệu quả các trang web lừa đảo mới (zero-day) nhờ các đặc trưng mang tính suy luận.
- Độ chính xác cao nhờ khả năng kết hợp nhiều cây quyết định và tối ưu hàm mục tiêu.
- Khả năng mở rộng tốt, có thể bổ sung thêm các đặc trưng mới khi cần thiết.

- Phù hợp cho triển khai thực tế trong các hệ thống phát hiện phishing tự động.

2.4. Phương pháp đánh giá

Để đánh giá hiệu quả của mô hình phát hiện trang web lừa đảo, bài toán sử dụng các chỉ số đánh giá được xây dựng dựa trên ma trận nhầm lẫn (confusion matrix). Ma trận nhầm lẫn là công cụ phổ biến trong học máy, dùng để so sánh nhãn dự đoán của mô hình với nhãn thực tế, qua đó phản ánh mức độ phân loại đúng và sai của mô hình đối với từng lớp dữ liệu.

Trong bài toán phân loại nhị phân, ma trận nhầm lẫn bao gồm bốn thành phần cơ bản:

- **True Positive (TP):** Số lượng trang web lừa đảo được mô hình dự đoán đúng là lừa đảo.
- **True Negative (TN):** Số lượng trang web hợp pháp được mô hình dự đoán đúng là hợp pháp.
- **False Positive (FP):** Số lượng trang web hợp pháp bị mô hình phân loại nhầm thành lừa đảo.
- **False Negative (FN):** Số lượng trang web lừa đảo bị mô hình phân loại nhầm thành hợp pháp.

Dựa trên các giá trị TP, TN, FP và FN, các chỉ số đánh giá hiệu suất mô hình được xác định như sau:

- **Tỷ lệ dương tính thực sự (True Positive Rate - TPR):** Tính tỷ lệ các trang web lừa đảo được xác định đúng so với tổng số trang web lừa đảo và còn được gọi là Recall (độ thu hồi).

$$Recall = TPR = \frac{TP}{TP + FN}$$

- **Tỷ lệ dương tính giả (False Positive Rate - FPR):** Tính tỷ lệ các trang web hợp pháp bị phân loại sai thành lừa đảo so với tổng số trang web hợp pháp.

$$FPR = \frac{FP}{TN + FP}$$

- **Độ chính xác (Accuracy):** Tỷ lệ dự đoán chính xác tổng thể được đo bằng độ chính xác

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

- **Độ chính xác của dự đoán dương tính (Precision):** Tính tỷ lệ các trang web lừa đảo được xác định chính xác so với tất cả các trang web được xác định là lừa đảo.

$$Precision = \frac{TP}{TP + FP}$$

- **F1-Score:** Là trung bình điều hòa của Precision và Recall.

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

2.5. Kết luận chương 2

Chương 2 đã trình bày phương pháp xây dựng mô hình phát hiện trang web lừa đảo dựa trên tập đặc trưng lai, kết hợp giữa đặc trưng URL và đặc trưng siêu liên kết, cùng với kỹ thuật học máy có giám sát. Quá trình xây dựng bộ dữ liệu, trích xuất và lựa chọn đặc trưng được thực hiện có hệ thống, nhằm phản ánh cả đặc điểm cấu trúc URL và hành vi của trang web thông qua cây DOM.

Trên cơ sở đó, thuật toán XGBoost được lựa chọn để xây dựng mô hình phân loại nhờ khả năng xử lý tốt các mối quan hệ phi tuyến, tập đặc trưng nhiều chiều và hạn chế hiện tượng quá khớp. Đồng thời, các chỉ số đánh giá hiệu năng như Accuracy, Precision, Recall và F1-score được sử dụng để đảm bảo việc đánh giá mô hình một cách khách quan.

Nhìn chung, chương này đã thiết lập nền tảng phương pháp luận cho mô hình phát hiện trang web lừa đảo được đề xuất, làm cơ sở cho việc triển khai thực nghiệm và đánh giá kết quả trong chương tiếp theo.

CHƯƠNG 3. TRIỂN KHAI THỰC NGHIỆM

3.1. Kịch bản

Trong thực nghiệm này, mô hình phát hiện trang web lừa đảo được xây dựng dựa trên tập đặc trưng lai, bao gồm các đặc trưng trích xuất từ cấu trúc URL và các đặc trưng dựa trên siêu liên kết trong mã nguồn HTML của trang web. Từ các đặc trưng này, một mô hình học máy có giám sát sử dụng thuật toán XGBoost được huấn luyện nhằm phân loại trang web thành hai lớp: hợp pháp và lừa đảo. Sau khi hoàn tất quá trình huấn luyện và đánh giá, mô hình thu được được triển khai trong một ứng dụng web mô phỏng, cho phép người dùng nhập vào một URL bất kỳ và hệ thống sẽ tự động trích xuất đặc trưng, áp dụng mô hình đã học để dự đoán mức độ lừa đảo của URL. Việc triển khai này nhằm minh họa khả năng ứng dụng thực tế của mô hình trong việc hỗ trợ phát hiện sớm các trang web lừa đảo.

3.2. Môi trường thực nghiệm

Trong chuyên đề này, thực nghiệm được triển khai trên môi trường máy tính cá nhân và sử dụng ngôn ngữ lập trình Python do tính linh hoạt và khả năng hỗ trợ mạnh mẽ cho các bài toán học máy.

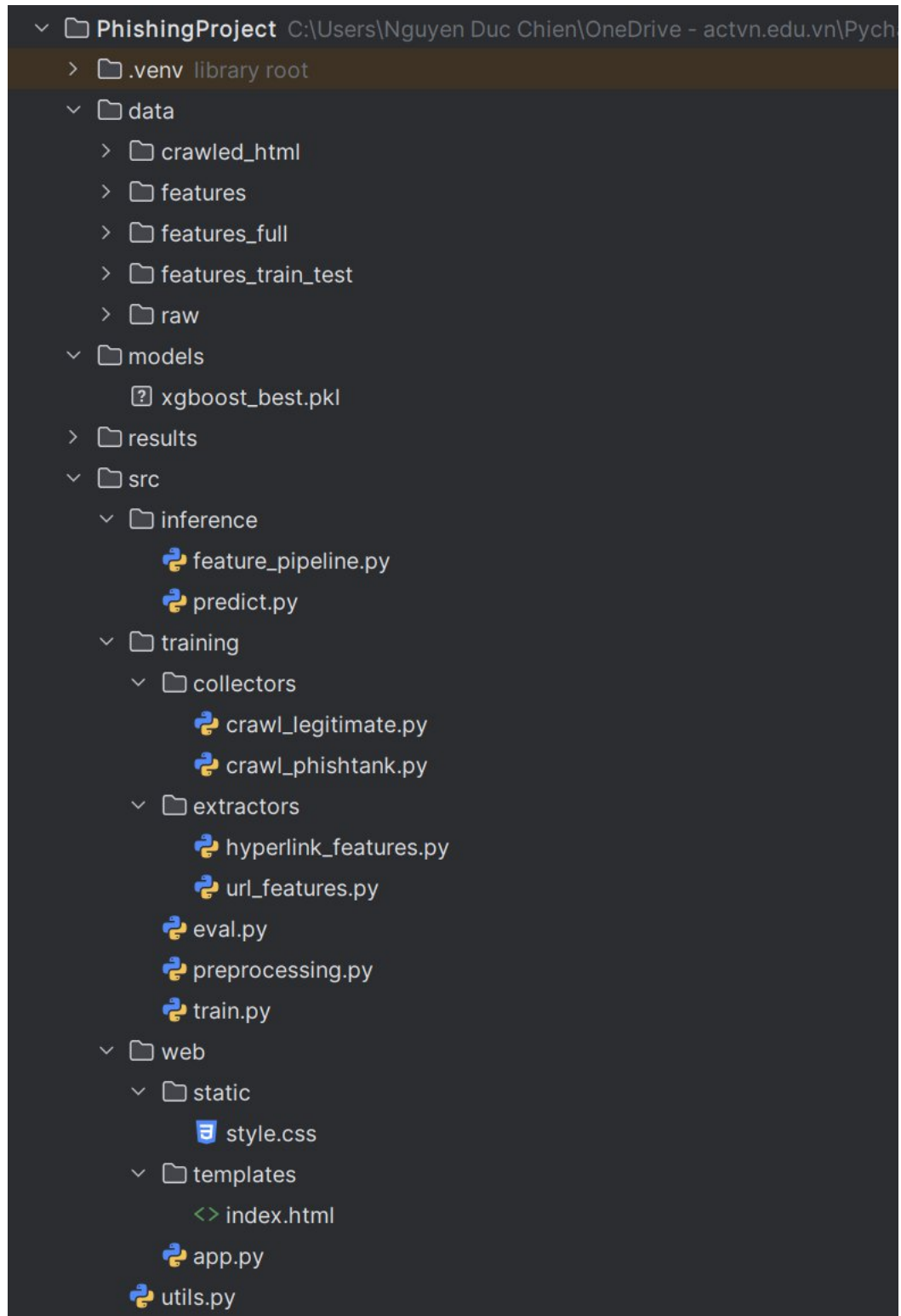
Công cụ sử dụng lập trình: PyCharm 2025.

Ngôn ngữ lập trình : Python.

Một số thư viện được sử dụng trong bài toán:

- csv, pandas, numpy: Xử lý dữ liệu
- scikit-learn: Chia dữ liệu và đánh giá
- xgboost: Huấn luyện mô hình
- BeautifulSoup: Thu thập và phân tích HTML

3.3. Mô tả cấu trúc thư mục hệ thống



Hình 3.1. Cấu trúc thư mục hệ thống

Quá trình triển khai thực nghiệm mô hình phát hiện trang web lừa đảo được xây dựng theo kiến trúc tách biệt giữa giai đoạn huấn luyện (offline) và giai đoạn dự đoán (online). Toàn bộ hệ thống được tổ chức theo cấu trúc thư mục được trình bày như Hình 3.1, nhằm đảm bảo tính rõ ràng, dễ mở rộng và thuận tiện cho việc đánh giá mô hình.

Giai đoạn 1: Thu thập và chuẩn bị dữ liệu (Training – Offline)

Thư mục data/ được sử dụng chỉ cho mục đích huấn luyện mô hình.

Trong đó:

- raw/ chứa danh sách URL thô. Danh sách được lưu dưới định dạng file .csv, đây là định dạng văn bản đơn giản dùng để lưu trữ dữ liệu dạng bảng, trong đó mỗi dòng tương ứng với một mẫu dữ liệu và mỗi cột tương ứng với một đặc trưng. Định dạng này giúp quá trình đọc, xử lý và huấn luyện mô hình Machine Learning được thực hiện nhanh chóng và thuận tiện. Danh sách bao gồm:
 - phishing_urls.csv: Các URL lừa đảo thu thập từ PhishTank
 - legitimate_urls.csv: Các URL hợp pháp thu thập từ Mendeley Data
- crawled_html/ lưu trữ mã nguồn HTML của các trang web hợp pháp và lừa đảo tương ứng, phục vụ cho việc trích xuất đặc trưng siêu liên kết.
- features/features_train.csv là tập dữ liệu cuối cùng sau khi trích xuất đặc trưng, gồm 25 đặc trưng lai và nhãn phân loại.

Các script trong src/training/collectors/ chịu trách nhiệm thu thập URL và tải mã nguồn HTML. Sau đó, các module trong extractors/ lần lượt trích xuất:

- Đặc trưng dựa trên URL,
- Đặc trưng dựa trên siêu liên kết (hyperlink) thông qua phân tích cây DOM.

Dữ liệu sau khi trích xuất được tiền xử lý tại preprocessing.py trước khi đưa vào huấn luyện.

Giai đoạn 2: Huấn luyện và đánh giá mô hình

Script train.py sử dụng tập đặc trưng đã chuẩn hóa để huấn luyện mô hình XGBoost theo bài toán phân loại nhị phân.

Sau khi huấn luyện, mô hình tốt nhất được lưu tại thư mục models/ dưới dạng xgboost_best.pkl.

Hiệu năng mô hình được đánh giá thông qua script `eval.py`, với các chỉ số Accuracy, Precision, Recall, F1-score và FPR.

Các kết quả đánh giá được lưu trong thư mục `results/` để phục vụ cho việc tổng hợp và trình bày trong báo cáo.

Giai đoạn 3: Dự đoán và triển khai trực tuyến (Inference – Online)

Để mô phỏng quá trình sử dụng trong thực tế, hệ thống dự đoán được tách riêng tại thư mục `src/inference/`.

- `feature_pipeline.py`: Thực hiện toàn bộ quy trình trích xuất 25 đặc trưng từ một URL đầu vào, bao gồm cả URL-based và hyperlink-based features.
- `predict.py`: tải mô hình XGBoost đã huấn luyện, nhận vector đặc trưng và đưa ra kết quả dự đoán (phishing hoặc legitimate).

Cách tổ chức này đảm bảo rằng quá trình dự đoán online hoàn toàn độc lập với dữ liệu huấn luyện, giúp mô hình có thể áp dụng cho các URL mới chưa từng xuất hiện.

Giai đoạn 4: Xây dựng giao diện web minh họa

Để trực quan hóa kết quả, một ứng dụng web đơn giản được xây dựng trong thư mục `src/web/`.

- `app.py` đóng vai trò backend (Flask/Django), tiếp nhận URL từ người dùng.
- `index.html` cung cấp giao diện nhập URL.
- Hệ thống trả về kết quả phân loại trang web ngay sau khi mô hình xử lý.

Phần web này nhằm minh họa khả năng ứng dụng thực tế của mô hình phát hiện trang web lừa đảo.

3.4. Triển khai huấn luyện mô hình XGBoost

Quá trình huấn luyện mô hình phát hiện trang web lừa đảo được triển khai thông qua script `train.py`. Script này chịu trách nhiệm thực hiện toàn bộ quá trình huấn luyện, bao gồm: đọc dữ liệu đặc trưng, chia tập huấn luyện – kiểm thử, huấn luyện mô hình XGBoost và lưu trữ mô hình đã huấn luyện để phục vụ cho giai đoạn dự đoán.

3.4.1. Chuẩn bị và chia tập dữ liệu

Dữ liệu đầu vào của quá trình huấn luyện là tập đặc trưng lai được lưu dưới dạng tệp CSV, trong đó mỗi mẫu dữ liệu tương ứng với một URL và bao gồm các

đặc trưng dựa trên URL, đặc trưng siêu liên kết (hyper link) và nhãn phân loại. Nhãn label nhận giá trị 0 đối với trang web hợp pháp và 1 đối với trang web lừa đảo.

Tập dữ liệu được chia thành hai phần:

- Tập huấn luyện (80%) dùng để huấn luyện mô hình
- Tập kiểm thử (20%) dùng để đánh giá hiệu năng mô hình.

Việc chia tập được thực hiện theo phương pháp *stratified split*, nhằm đảm bảo tỷ lệ giữa hai lớp dữ liệu được giữ nguyên trong cả hai tập. Tham số *random_state* được cố định để đảm bảo khả năng tái lập kết quả thực nghiệm. Sau khi chia, các tập dữ liệu huấn luyện và kiểm thử được lưu lại để phục vụ cho các bước xử lý tiếp theo.

```
data = pd.read_csv(hybrid_features_csv)
X = data.drop(columns=["label"])
y = data["label"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=99
)
```

3.4.2. Huấn luyện và tối ưu mô hình XGBoost

Mô hình học máy được sử dụng trong chuyên đề là XGBoost, với cấu hình cho bài toán phân loại nhị phân thông qua hàm mục tiêu *binary:logistic*. Để lựa chọn được bộ tham số phù hợp, phương pháp Grid Search kết hợp với Cross-Validation được áp dụng trong quá trình huấn luyện.

Các tham số chính của mô hình được khảo sát bao gồm:

- Số lượng cây (*n_estimators*).
- Độ sâu tối đa của cây (*max_depth*).
- Tốc độ học (*learning_rate*).
- Tỷ lệ lấy mẫu dữ liệu (*subsample*).
- Tỷ lệ lấy mẫu đặc trưng (*colsample_bytree*).

```
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.05, 0.1],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0],
}
```

Quá trình đánh giá tham số được thực hiện bằng phương pháp Stratified K-Fold Cross-Validation với 5 lần chia, nhằm đảm bảo tính ổn định và giảm thiểu ảnh hưởng của sự mất cân bằng dữ liệu. Sau khi hoàn tất quá trình huấn luyện, mô hình có hiệu năng tốt nhất được lựa chọn và lưu trữ dưới dạng tệp để phục vụ cho giai đoạn dự đoán.

```
xgb = XGBClassifier(  
    objective="binary:logistic",  
    eval_metric="logloss",  
    random_state=99,  
    n_jobs=8,  
    use_label_encoder=False,  
)  
  
cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=99  
)  
  
grid = GridSearchCV(  
    estimator=xgb,  
    param_grid=param_grid,  
    cv=cv,  
    n_jobs=8,  
    verbose=1  
)  
  
grid.fit(X_train, y_train)
```

Sau khi quá trình huấn luyện hoàn tất, mô hình có hiệu năng tốt nhất được lựa chọn và lưu trữ để sử dụng trong giai đoạn dự đoán.

```
best_model = grid.best_estimator_  
save_model(best_model, "models/xgboost_best.pkl")
```

3.5. Đánh giá mô hình

Bảng 3.1. Ma trận nhầm lẫn

	Dự đoán (nhãn 0)	Dự đoán (nhãn 1)
Thực tế (nhãn 0)	2857	163
Thực tế (nhãn 1)	161	2859

Trong đó:

- **True Positive (TP) = 2857:** Số lượng trang web lừa đảo được mô hình dự đoán đúng là lừa đảo.

- **True Negative (TN) = 2859**: Số lượng trang web hợp pháp được mô hình dự đoán đúng là hợp pháp.
- **False Positive (FP) = 163**: Số lượng trang web hợp pháp bị mô hình phân loại nhầm thành lừa đảo.
- **False Negative (FN) = 161**: Số lượng trang web lừa đảo bị mô hình phân loại nhầm thành hợp pháp.

Bảng 3.1. Đánh giá hiệu suất mô hình

	Kết quả
Accuracy	0.9464
Precision	0.9461
Recall (TPR)	0.9467
F1-score	0.9464
FPR	0.0540

Trong đó:

- **Accuracy**: tỷ lệ dự đoán đúng trên toàn bộ tập kiểm thử.
- **Precision**: mức độ chính xác của các dự đoán trang web lừa đảo.
- **Recall (TPR)**: khả năng phát hiện đúng các trang web lừa đảo.
- **F1-score**: chỉ số tổng hợp giữa Precision và Recall.
- **FPR**: tỷ lệ trang web hợp pháp bị phân loại nhầm là lừa đảo.

3.6. Ứng dụng web mô phỏng

Để minh họa khả năng ứng dụng thực tế của mô hình phát hiện trang web lừa đảo, một ứng dụng web mô phỏng đã được xây dựng và tích hợp với mô hình XGBoost đã huấn luyện. Ứng dụng này cho phép người dùng nhập vào một URL bất kỳ và nhận kết quả phân loại dựa trên mô hình học máy.

Khi người dùng gửi một URL cần kiểm tra, hệ thống sẽ tự động thực hiện toàn bộ quá trình dự đoán, bao gồm: trích xuất các đặc trưng dựa trên cấu trúc URL, tải và phân tích mã nguồn HTML để trích xuất các đặc trưng siêu liên kết, sau đó kết hợp các đặc trưng để tạo vector đặc trưng đầu vào cho mô hình. Mô hình XGBoost đã được huấn luyện trước đó được nạp và sử dụng để đưa ra kết quả dự đoán tương ứng.

Kết quả phân loại được hiển thị trực tiếp trên giao diện web, cho phép người dùng nhanh chóng đánh giá mức độ an toàn của URL trước khi truy cập. Việc xây dựng ứng dụng web trong chuyên đề này không nhằm mục đích phát triển một sản

phẩm hoàn chỉnh, mà tập trung vào việc mô phỏng quy trình triển khai và chứng minh khả năng tích hợp mô hình phát hiện lừa đảo trong các hệ thống thực tế.

Phishing Website Detection

URL-based & Hyperlink-based Detection using XGBoost

Enter URL to check...

Check URL

Detection Result

URL	https://www.facebook.com/home.php
Prediction	Legitimate
Phishing Probability	0.4471

Hình 3.2. Giao diện web khi dự đoán URL hợp pháp

Phishing Website Detection

URL-based & Hyperlink-based Detection using XGBoost

Check URL

Detection Result

URL	https://millennium-app.info/8EEZGX JSFFMA8J2VQVYC4V...	
Prediction	Phishing	
Phishing Probability	0.8995	

Hình 3.3. Giao diện web khi dự đoán URL lừa đảo

3.7. Kết luận chương 3

Chương 3 đã trình bày quá trình triển khai thực nghiệm mô hình phát hiện trang web lừa đảo dựa trên đặc trưng lai bằng kỹ thuật học máy. Từ việc xây dựng kịch bản thực nghiệm, lựa chọn môi trường triển khai cho đến thiết kế kiến trúc hệ thống và quy trình thực thi, mô hình được hiện thực hóa một cách đầy đủ và có tính ứng dụng.

Kết quả cho thấy mô hình XGBoost kết hợp tập đặc trưng URL và siêu liên kết có thể được triển khai hiệu quả trong môi trường thực tế, với khả năng phát hiện các URL mới thông qua pipeline dự đoán độc lập. Việc xây dựng ứng dụng web mô phỏng đã minh họa rõ ràng tính khả thi của mô hình, đồng thời mở ra hướng phát triển cho các hệ thống phát hiện phishing tự động trong tương lai.

KẾT LUẬN

Chương 1 đã hệ thống lại những kiến thức tổng quan về trang web lừa đảo và các kỹ thuật phát hiện hiện có. Chương này cũng đã chỉ ra rằng để đảm bảo khả năng phát hiện hiệu quả các cuộc tấn công mới (zero-day) thì không thể chỉ dựa vào các phương pháp truyền thống như danh sách đen (blacklist) hay so sánh giao diện, mà cần phải sử dụng các kỹ thuật học máy với khả năng suy luận từ dữ liệu.

Chương 2 đã tập trung xây dựng và đề xuất mô hình phát hiện dựa trên đặc trưng lai. Đó là sự kết hợp giữa nhóm đặc trưng dựa trên cấu trúc URL và nhóm đặc trưng dựa trên siêu liên kết được trích xuất từ mã nguồn trang web. Qua đó, thuật toán XGBoost đã được lựa chọn làm nòng cốt cho mô hình phân loại nhờ vào khả năng xử lý tốt các dữ liệu phi tuyến và hiệu năng vượt trội so với các thuật toán truyền thống. Tuy nhiên, như là quy luật của sự phát triển công nghệ, các kỹ thuật tấn công lừa đảo luôn thay đổi khôn lường, đòi hỏi các đặc trưng và mô hình cũng cần được tinh chỉnh liên tục để đối phó với những thủ đoạn mới.

Trong chương 3, một chương trình minh họa hoạt động của mô hình phát hiện trang web lừa đảo đã được xây dựng thành công. Sở dĩ việc xây dựng ứng dụng web demo được thực hiện là để chứng minh tính khả thi của việc tích hợp mô hình học máy vào quy trình kiểm tra an toàn thông tin thực tế, thay vì chỉ dừng lại ở các kết quả đánh giá trên giấy. Điều đó giúp người dùng có công cụ trực quan để đánh giá mức độ tin cậy của một đường dẫn trước khi truy cập.

Dù vậy, vẫn còn một số vấn đề liên quan đến khả năng bao quát của mô hình vẫn chưa được giải quyết triệt để trong chuyên đề. Trước hết, chuyên đề này mới chỉ đề cập đến việc sử dụng các đặc trưng về URL và siêu liên kết mà chưa khai thác sâu các đặc trưng về nội dung văn bản hay xử lý hình ảnh giao diện trang web. Việc đánh giá mô hình cũng chỉ mới được thực hiện trên tập dữ liệu tĩnh được thu thập từ trước mà chưa được kiểm thử rộng rãi trong môi trường mạng thời gian thực với lưu lượng lớn. Và chuyên đề này vẫn chưa làm rõ được hết các kịch bản tấn công mà kẻ xấu sử dụng kỹ thuật che giấu mã nguồn để qua mặt bộ trích xuất đặc trưng. Việc giải quyết những điểm tồn tại này cũng chính là hướng phát triển tương lai của chuyên đề.

TÀI LIỆU THAM KHẢO

- [1]. Sumitra Das Guptta et al. *Modeling Hybrid Feature-Based Phishing Websites Detection Using Machine Learning Techniques*, Annals of Data Science, Springer, 2022.
- [2]. PhishTank. *PhishTank Open Source Database*. URL: <https://phishtank.org/> (Accessed: 15/12/2025)
- [3]. Potpelwar, R. LegitPhish Dataset. *Mendeley Data*, V1, Apr. 7 2025. URL: <https://data.mendeley.com/datasets/hx4m73v2sf/1> (Accessed: 15/12/2025)
- [4]. J. K. S. Kaitholikkal and A. B., *Phishing URL dataset*, Mendeley Data, V1, Apr. 2 2024. URL: <https://data.mendeley.com/datasets/vfszbj9b36/1> (Accessed: 15/12/2025)
- [5]. T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [6]. J. H. Friedman. Greedy function approximation: A gradient boosting machine, *Annals of Statistics*, vol. 29, no. 5, 2001.
- [7]. M. Al-Sarem et al. An Optimized XGBoost Model for Phishing Websites Detection, *Electronics*, vol. 10, 2021.
- [8]. A. K. Jain and B. B. Gupta. Phishing Detection: Analysis of Visual Similarity Based Approaches, *Security and Privacy*, 2018.
- [9]. S. Sahingoz et al. Machine learning based phishing detection from URLs, *IEEE Access*, vol. 7, 2019.

PHỤ LỤC

Phụ lục A: Mã nguồn trích xuất đặc trưng URL

```
if __name__ == "__main__":
    main()
from urllib.parse import urlparse
import tldextract
import os
import time
from pathlib import Path
from src.utils import ensure_dir, logging
import csv
import ipaddress

ROOT = Path(__file__).resolve().parents[3]

def read_csv(csv_path):
    rows = []
    with open(csv_path, newline='', encoding='utf-8') as file_csv:
        reader = csv.DictReader(file_csv)
        for row in reader:
            if 'url' in row and row['url'].strip():
                rows.append({
                    'url': row['url'].strip(),
                })
    return rows

def is_valid_ip(ip_str):
    try:
        ipaddress.ip_address(ip_str)
        return True
    except ValueError:
        return False

def extract_url_features(url):
    parsed = urlparse(url if url.startswith(("http://", "https://")) else "https://" + url)
    scheme = parsed.scheme or ""
    domain_name = parsed.hostname or ""
    path = parsed.path or ""
    full_url = url

    ext = tldextract.extract(domain_name)
    subdomain = ext.subdomain or ""
    domain = ext.domain or ""
    suffix = ext.suffix or ""
    base_domain = f"{domain}.{suffix}" if domain and suffix else ""
    subdomain_domain = f"{subdomain}.{domain}" if subdomain and domain else ""

    f = {}

    # UF2: Count subdomain in URL
    dot_count = subdomain_domain.count('.')
    if dot_count > 2:
        f['UF2'] = 1
    elif dot_count == 2:
        f['UF2'] = 0.5
    else:
        f['UF2'] = 0

    # UF3: IP Address is domain name
```

```

f['UF3'] = int(is_valid_ip(domain_name))

# UF4: "@" symbol in URL
f['UF4'] = int('@' in full_url)

# UF5: Length of URL
url_len = len(full_url)
if url_len < 75:
    f['UF5'] = 0
elif url_len < 100:
    f['UF5'] = 0.5
else:
    f['UF5'] = 1

# UF6: Depth of URL
f['UF6'] = len([p for p in path.split('/') if p])

# UF7: Redirection '//' in URL
pos = full_url.find('//', full_url.find('//') + 2)
f['UF7'] = int(pos > 7)

# UF8: http/https in domain name
f['UF8'] = int('http' in domain_name.lower() or 'https' in
domain_name.lower())

# UF9: HTTPS in scheme
f['UF9'] = int(scheme.lower() != 'https')

# UF10: Using URL shortening service "tinyURL"
shortlist =
['bit.ly', 'bitly.com', 'bitly.is', 'j.mp', 'tinyurl.com', 't.co', 'goo.gl', 'ow.ly',
'buff.ly', 'campsite.bio']
f['UF10'] = int(any(s in domain_name.lower() for s in shortlist))

# UF11: Prefix or Suffix "-" in domain name
f['UF11'] = int('-' in domain_name)

# UF12: Existence of sensitive word
sensitive_words = ['account', 'bank', 'login', 'signin', 'update',
'verify', 'validation', 'confirm', 'activate']

f['UF12'] = int(any(w in full_url.lower() for w in sensitive_words))

# UF13: Existence of trendy brand name
brands =
['microsoft', 'google', 'apple', 'amazon', 'adobe', 'linkedin', 'facebook', 'meta', '
whatsapp', 'spotify']
f['UF13'] = int(any(b in full_url.lower() for b in brands))

# UF14: Existence of upper case letter
f['UF14'] = int(any(c.isupper() for c in full_url))

# UF15: Number of dots in URL
f['UF15'] = int(full_url.count('.') > 2)

return f

def save_extracted_urls(path):
    extracted = set()
    if not os.path.exists(path):
        return extracted
    with open(path, 'r', newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:

```



```

        url = row.get('url')
        if url and url.strip():
            extracted.add(url.strip())
    return extracted

def main(csv_in=None, path_save_uf=None, delay=0.5):
    csv_in = csv_in or os.path.join(ROOT, "data", "raw", "phishing_urls.csv")
    path_save_uf = path_save_uf or os.path.join(ROOT, "data", "features",
"phishing_url_features.csv")

    ensure_dir(os.path.dirname(path_save_uf))

    rows = read_csv(csv_in)
    logging.info(f"Loaded {len(rows)} URLs")

    extracted_urls = save_extracted_urls(path_save_uf)
    logging.info(f"Found {len(extracted_urls)} URLs already extracted")

    FIELDNAMES = ['url'] + [f'UF{i}' for i in range(2, 16)] + ['label']
    file_is_empty = (not os.path.exists(path_save_uf)) or
os.path.getsize(path_save_uf) == 0

    with open(path_save_uf, 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=FIELDNAMES)
        if file_is_empty:
            writer.writeheader()

    for r in rows:
        url = r['url']

        if url in extracted_urls:
            logging.info(f"Skip (already extracted): {url}")
            continue

        try:
            features = extract_url_features(url)

            row = {'url': url}
            for i in range(2, 16):
                row[f'UF{i}'] = features.get(f'UF{i}', 0)
            row['label'] = 1

            writer.writerow(row)

            extracted_urls.add(url)
            logging.info(f"Extracted features successfully for URL:
{url}")

            time.sleep(delay)
        except Exception as e:
            logging.error(f"Extraction of failure features: {url} | {e}")

    logging.info(f"Saved URL features to: {path_save_uf}")

if __name__ == "__main__":
    main()

```

Phụ lục B: Mã nguồn trích xuất đặc trưng siêu liên kết (hyperlink)

```

from bs4 import BeautifulSoup
from urllib.parse import urlparse, urljoin
from collections import Counter
from pathlib import Path
from src.utils import ensure_dir, logging

```

```

import tldextract
import time
import os
import csv

ROOT = Path(__file__).resolve().parents[3]

def read_html_and_url(filepath):
    url = None
    html_content = ""

    with open(filepath, "r", encoding="utf-8") as f:
        lines = f.readlines()

    if len(lines) >= 2 and lines[0].strip().startswith("<!-- url:"):
        url_line = lines[0].strip()
        url = url_line[len("<!-- url:"):].strip().rstrip('\n').strip()
        html_content = "".join(lines[2:])
    else:
        html_content = "".join(lines)

    return url, html_content

def is_external(href, base_domain):
    try:
        h = urlparse(href)
        hostname = h.hostname
        ext = tldextract.extract(hostname)
        domain = ext.domain or ""
        suffix = ext.suffix or ""
        href_base_domain = f"{domain}.{suffix}" if domain and suffix else ""
        return href_base_domain != base_domain
    except Exception:
        return True

def extract_hyperlink_features(html_text, url):
    soup = BeautifulSoup(html_text, "lxml")
    parsed = urlparse(url if url.startswith(("http", "https")) else "http://" + url)
    domain_name = parsed.hostname

    ext = tldextract.extract(domain_name)
    domain = ext.domain or ""
    suffix = ext.suffix or ""
    base_domain = f"{domain}.{suffix}" if domain and suffix else ""

    anchors = soup.find_all('a')
    forms = soup.find_all('form')
    links = soup.find_all('link')

    total_anchors = len(anchors)

    out_anchors = 0
    null_anchors = 0

    href_list = []

    for a in anchors:
        href = (a.get('href') or "").strip()
        if href in [ "", "#", "#content" ] or href.lower().startswith("javascript"):
            null_anchors += 1
        if href:
            full_href = urljoin(url, href)

```

```

        href_list.append(full_href)
        if is_external(full_href, base_domain):
            out_anchors += 1

f = {}

# HF1: No hyperlink
total_hyperlink = total_anchors
f['HF1'] = int(total_hyperlink == 0)

# HF2: Internal hyperlink ratio
in_anchors = total_anchors - out_anchors - null_anchors
internal_ratio = in_anchors / total_anchors if total_anchors > 0 else 0
f['HF2'] = 0 if internal_ratio >= 0.5 else 1

# HF3: External hyperlink ratio
external_ratio = out_anchors / total_anchors if total_anchors > 0 else 0
f['HF3'] = 1 if external_ratio > 0.5 else 0

# HF4: Internal/ External CSS
f['HF4'] = int(any(
    l.get('rel') and 'stylesheet' in l.get('rel') and
    l.get('href') and is_external(urljoin(url, l.get('href'))),
base_domain)
    for l in links
))

# HF5: Form action
f['HF5'] = 0
for form in forms:
    action = (form.get('action') or "").strip().lower()
    if action in ["", "#", "javascript:void(0)"]:
        f['HF5'] = 1
        break
    if action.endswith(".php"):
        f['HF5'] = 1
        break
    if is_external(urljoin(url, action), base_domain):
        f['HF5'] = 1
        break

# HF6: Null hyperlink
null_ratio = null_anchors / total_anchors if total_anchors > 0 else 0
f['HF6'] = 1 if null_ratio > 0.34 else 0

# HF7: External favicon
f['HF7'] = 0
for l in links:
    rel = " ".join(l.get('rel', []))
    if 'icon' in rel.lower():
        href = l.get('href') or ""
        if href and is_external(urljoin(url, href), base_domain):
            f['HF7'] = 1
            break

# HF8: Common page ratio
f['HF8'] = 0
if total_anchors > 0 and href_list:
    most_common = Counter(href_list).most_common(1)[0][1] # đếm và trả
lớn nhất
    f['HF8'] = most_common / total_anchors

# HF9: Common page in footer
footer = soup.find('footer')

```

```

f['HF9'] = 0
if footer:
    footer_links = [urljoin(url, a.get('href'))
                     for a in footer.find_all('a') if a.get('href')]
    if footer_links:
        most_common_footer = Counter(footer_links).most_common(1)[0][1]
        f['HF9'] = most_common_footer / len(footer_links)

# HF10: SFH
f['HF10'] = 0
for form in forms:
    action = (form.get('action') or "").strip().lower()
    if action in ["", "about:blank"]:
        f['HF10'] = 1
        break
    if is_external(urljoin(url, action), base_domain):
        f['HF10'] = max(f['HF10'], 0.5)

return f

def save_extracted_urls(path):
    extracted = set()
    if not os.path.exists(path):
        return extracted
    with open(path, 'r', newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            url = row.get('url')
            if url and url.strip():
                extracted.add(url.strip())
    return extracted

def main(folder_save_html=None, path_save_hf=None, delay=0.5):
    folder_save_html = folder_save_html or os.path.join(ROOT, "data",
    "crawled_html", "phishing")
    path_save_hf = path_save_hf or os.path.join(ROOT, "data", "features",
    "phishing_hyperlink_features.csv")

    ensure_dir(os.path.dirname(path_save_hf))

    files = [f for f in os.listdir(folder_save_html) if f.endswith(".html")]

    extracted_urls = save_extracted_urls(path_save_hf)
    logging.info(f"Found {len(extracted_urls)} URLs already extracted")

    FIELDNAMES = ['url'] + [f'HF{i}' for i in range(1, 11)] + ['label']
    file_is_empty = (not os.path.exists(path_save_hf)) or
    os.path.getsize(path_save_hf) == 0

    with open(path_save_hf, 'a', newline='', encoding='utf-8') as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=FIELDNAMES)
        if file_is_empty:
            writer.writeheader()

    for f in files:
        filepath = os.path.join(folder_save_html, f)
        url, html = read_html_and_url(filepath)

        if url in extracted_urls or not url or not html.strip():
            logging.info(f"Skip (already extracted): {url}")
            continue

        try:
            features = extract_hyperlink_features(html, url)

```

```

        row = {'url': url,}
        for i in range(1, 11):
            row[f'HF{i}'] = features.get(f'HF{i}', 0)
        row['label'] = 1

        writer.writerow(row)

        extracted_urls.add(url)
        logging.info(f"Successfully extracted HTML features of URL:
{url}")

        time.sleep(delay)
    except Exception as e:
        logging.error(f"Failed to extract HTML features of URL: {url}
| {e}")

    logging.info(f"Saved HTML features to: {path_save_hf}")

if __name__ == '__main__':
    main()

```

Phụ lục C: Mã nguồn huấn luyện mô hình

```

import os
from pathlib import Path
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold
from xgboost import XGBClassifier
from src.utils import ensure_dir, save_model, logging

ROOT = Path(__file__).resolve().parents[2]

def data_in(hybrid_features_csv=None, path_save_train_test=None,
test_size=0.2):
    hybrid_features_csv = hybrid_features_csv or os.path.join(ROOT, "data",
"features_full", "hybrid_features_full.csv")

    data = pd.read_csv(hybrid_features_csv)

    X = data.drop(columns=["label"])
    y = data["label"]

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=test_size,
        stratify=y,
        random_state=99
    )

    path_save_train_test = path_save_train_test or os.path.join(ROOT, "data",
"features_train_test")
    ensure_dir(path_save_train_test)

    pd.concat([X_train, y_train],
axis=1).to_csv(os.path.join(path_save_train_test, "train.csv"), index=False)
    pd.concat([X_test, y_test],
axis=1).to_csv(os.path.join(path_save_train_test, "test.csv"), index=False)

    logging.info("Train/Test datasets saved successfully.")

```

```

    return X_train, X_test, y_train, y_test

def train_xgboost(X_train, y_train):
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [3, 5, 7],
        "learning_rate": [0.05, 0.1],
        "subsample": [0.8, 1.0],
        "colsample_bytree": [0.8, 1.0],
    }

    xgb = XGBClassifier(
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=99,
        n_jobs=8,
        use_label_encoder=False,
    )

    cv = StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=99
    )

    grid = GridSearchCV(
        estimator=xgb,
        param_grid=param_grid,
        cv=cv,
        n_jobs=8,
        verbose=1
    )

    grid.fit(X_train, y_train)

    logging.info(f"Best parameters: {grid.best_params_}")

    return grid.best_estimator_, grid

def main():
    # data
    X_train, X_test, y_train, y_test = data_in()

    # Train model
    model, grid = train_xgboost(X_train, y_train)

    # Save model
    ensure_dir(os.path.join(ROOT, "models"))
    save_model(model, os.path.join(ROOT, "models", "xgboost_best.pkl"))

    # Save results
    ensure_dir(os.path.join(ROOT, "results"))

    pd.DataFrame(grid.cv_results_).to_csv(os.path.join(ROOT, "results",
"grid_results.csv"), index=False)

    logging.info("Training finished successfully.")

```

Phụ lục D: Mã nguồn đánh giá mô hình

```
import os
from pathlib import Path
import pandas as pd
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
)
from src.utils import ensure_dir, load_model, logging

ROOT = Path(__file__).resolve().parents[2]

model_path = os.path.join(ROOT, "models", "xgboost_best.pkl")
features_test_path = os.path.join(ROOT, "data", "features_train_test",
"test.csv")
result_path = os.path.join(ROOT, "results")

def compute_fpr(cm):
    tn, fp, fn, tp = cm.ravel()
    fpr = fp / (fp + tn) if (fp + tn) != 0 else 0.0
    return fpr

def evaluate():
    ensure_dir(result_path)
    model = load_model(model_path)
    logging.info("Model loaded successfully.")
    data = pd.read_csv(features_test_path)
    X_test = data.drop(columns=["label"])
    y_test = data["label"]
    y_pred = model.predict(X_test)
    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    fpr = compute_fpr(cm)
    pd.DataFrame(
        cm,
        index=["Actual_0", "Actual_1"],
        columns=["Pred_0", "Pred_1"]
    ).to_csv(os.path.join(result_path, "confusion_matrix.csv"))
    # raw results
    raw_results = classification_report(y_test, y_pred, output_dict=True)
    pd.DataFrame(raw_results).transpose().to_csv(os.path.join(result_path,
"raw_results.csv"), index=False)
    summary_results = {
        "Accuracy": raw_results["accuracy"],
        "Precision": raw_results["1"]["precision"],
        "Recall (TPR)": raw_results["1"]["recall"],
        "F1-score": raw_results["1"]["f1-score"],
        "FPR": fpr
    }

    pd.DataFrame([summary_results]).to_csv(os.path.join(result_path,
"summary_results.csv"), index=False)

    print("----- Summary Results -----")
    for key, value in summary_results.items():
        print(f"{key}: {value:.4f}")

    logging.info("Evaluation finished successfully.")

if __name__ == "__main__":
    evaluate()
```

